

1. Introduction to Programming

- 1.1 What is a Program?
- 1.2 What Does a Programmer Do?
- 1.3 Programs vs Recipes
- 1.4 Real-Life Analogy: Importance of Instruction Order
- 1.5 Programming Reality
- 1.6 Understanding Computers
- 1.7 Activity: Instructing a Friend to Build Lego
- 1.8 Programs = English Instructions + Programming Language
- 1.9 Compilers vs Interpreters
- 1.10 Core Concepts in Programming
- 1.11 What Tools Do You Need to Program?
- 1.12 Terminal
 - 1.12.1 Command Line Interface (CLI) Basics
 - 1.12.2 Editing Code with VIM
 - 1.12.3 Writing Your First Python Program
 - 1.12.4 Using the Python Interpreter (REPL)
 - 1.12.5 The `print()` Function in Python
 - 1.12.6 Compiling Python to Bytecode

2. Variables and Data Types

- 1. Variables
 - 1.1 What is a Variable
 - 1.2 Variable Characteristics in Python
 - 1.3 Data Types and Objects
 - 1.4 Naming Rules for Variables
- 2. Data Types
 - 2.1 Integer (`int`)
 - 2.2 Float (`float`)
 - 2.3 Boolean (`bool`)
 - 2.4 String (`str`)
 - 2.5 Type Annotation
 - 2.6 Type Errors
- 3. Name of Different Parts of Statement
 - 3.1 literal, Variable, Operator, Expression
 - 3.2 Types of Operators
- 4. Type Casting
 - 4.1 Explicit Casting
 - 4.2 Implicit Casting
- 5. Input and Output
 - 5.1 `input()` Function
 - 5.2 Formatted Output
 - 5.3 Format Specifications

3. Conditionals

- 3.1 **Why Do We Need Decisions?**
- 3.2 **Control Flow Basics**
- 3.3 Simple Branching: `if` Statement
- 3.4 **Booleans and Comparison Operators**
- 3.5 Two-Way Branching: `if-else`
- 3.6 Multi-Way Branching: `if-elif-else`
- 3.7 Boolean Logic Operators
- 3.8 Compound Conditions
- 3.9 Nested `if` Statements
- 3.10 The `pass` Statement
- 3.11 Summary

4. Containers

- 4.1 List
 - 4.1.1 Concept

- 4.1.2 Initializing
- 4.1.3 Accessing and Indexing
- 4.1.4 Mutability
- 4.1.5 Common Methods
- 4.1.6 Slicing
- 4.1.7 Two-Dimensional Lists

4.2 Tuple

- 4.2.1 Concept
- 4.2.2 Initializing
- 4.2.3 Accessing Tuple
- 4.2.4 Immutability
- 4.2.5 Common Methods
- 4.2.6 Tuples in Functions
- 4.2.7 Packing and Unpacking
- 4.2.8 Converting Between Tuples and Lists

4.3 Dictionary

1. Concept
2. Initializing
3. Accessing Dictionary
4. Adding, Modifying, and Deleting
5. Iterating
6. Other Common Methods
7. Comparison With Other Data Structures

4.4 String

1. Concept
2. Common Operations
3. Common Built-in Methods
4. Conversions Between Strings and Other Data Types
5. Immutability

4.5 Mutable & Immutable

- 4.5.1 Concept
- 4.5.2 Mutable: List
- 4.5.3 Immutable: String
- 4.5.4 Another Immutable Example: Tuple

4.6 Command Line Arguments

1. Concept
2. Obtain Command Line Arguments
3. Pass Command Line Arguments

5. Loops

- 5.1 Why Loops
- 5.2 Classification of Loops
- 5.3 Control Flow: While
- 5.4 Control Flow: For
- 5.5 Infinite Loops
 - 5.5.1 Definition
 - 5.5.2 Common Examples
 - 5.5.3 How to Break or Avoid Infinite Loops

6. Flow and Functions

- 6.1 What is a Function?
- 6.2 Function Definition Syntax
- 6.3 Function Parameters and Return Values
- 6.4 Parameters vs Arguments
- 6.5 Returning Multiple Values (using tuples)
- 6.6 Why Use Functions?
- 6.7 Type Hinting
- 6.8 Docstrings
- 6.9 Variable Scope
- 6.10 **Pure vs Impure Functions**

- 6.11 Modules
- 6.12 Flow of Execution
- 6.13 The `main()` Function and `__name__`
- 6.14 Debugging with `pdb` (Python Debugger)
- 6.15 Informal Trace (Manual Variable Tracking)

7 Classes and Objects

- 7.1 Everything Is an Object in Python
- 7.2 Functions vs Methods
- 7.3 Defining Classes in Python
- 7.4 Creating Objects (Instantiation)
- 7.5 Instance Variables and Methods
- 7.6 Static Methods
- 7.7 Class Methods (Optional)
- 7.8 Comparison: Instance vs Class vs Static Methods
- 7.9 Special Methods: `__str__`, `__repr__` and `__eq__`
- 7.10 Class Variables vs Instance Variables
- 7.11 Module Import and Driver Programs
- 7.12 Getters and Setters

8 More Flow Control

- 8.1 Advanced Flow Control: `break`, `continue`, `return`, `raise`, `try-except`, `assert`
 - 8.1.1 `break`: Exit a loop early
 - 8.1.2 `continue`: Skip the current iteration
 - 8.1.3 `return`: End function and return value
 - 8.1.4 `raise`: Manually trigger an exception
 - 8.1.5 `try-except`: Handle exceptions gracefully
 - 8.1.6 `assert`
- 8.2 `for` Loops and the `range` Function
 - 8.2.1 `for` loop
 - 8.2.2 `range(start, stop, step)` function
- 8.3 Nested Structures and Combined Control Flow
 - 8.3.1 Nested loop with `break`
- 8.4 Built-in Exceptions You Can Raise in Python

9. File Input/Output

- 9.1 Introduction: Why is File I/O Important?
- 2. Standard Input/Output and File I/O in Python
- 3. Reading Files
 - 3.1 `read()`: Reads the entire file as a string
 - 3.2 `readline()`: Reads one line at a time
 - 3.3 `readlines()`: Reads all lines into a list
- 4. Writing Files
 - 4.1 `write()`: Writes a string to the file
 - 4.2 `writelines()`: Writes a list of strings
 - 4.3 Writing to a file with `print()`
- 5. Closing Files
- 6. File Paths
- 7. The `os` Module for File Operations
- 8. Exception Handling: `try-except-finally`
- String Methods: `split()` and `strip()`
 - 1. `split()` Method
 - 2. `strip()` Method

Note: `strip()` matches **individual characters**, not substrings

10. Testing

- 10.1 Testing Fundamentals
- 10.2 Designing Test Cases
- 10.3 `assert` Statement & Manual Testing
- 10.4 Writing a Test Driver
- 10.5 The `unittest` Framework

10.6 Using `doctest` Module

10.7 **Testing Terminology**

10.8 White Box and Black Box Testing

11 Recursion

11.1 What is Recursion?

11.2 Two Core Parts of Recursion

11.3 Problems Solved Using Recursion

11.4 Different Ways to Write Base Cases

11.5 Infinite Recursion

11.6 Understanding Memory Use in Recursion: The Call Stack

11.7 Tower of Hanoi Problem

12. Multi-dimensional Lists and Numpy Arrays

12.1 Review: Python Lists

12.2 Multi-dimensional Lists

12.3 2D Lists

12.4 3D Lists

12.5 NumPy Basics

12.6 NumPy Array Operations

12.7 Traversing with NumPy

12.8 Practice Activity: Matrix Game Simulation

1. Introduction to Programming

1.1 What is a Program?

- A **program** is a **sequence of instructions** that a computer executes to perform a specific task.
- The instructions must be **precise and logically ordered**.

Example in Python:

```
1 # A simple program: calculate the sum of two numbers
2 a = 5
3 b = 3
4 sum = a + b
5 print(sum) # Output: 8
```

1.2 What Does a Programmer Do?

- A **programmer** writes the instructions that make up computer programs.
- Code must be readable by others and should be explainable.

```
1 # Calculate the area of a circle
2 radius = 2.5
3 pi = 3.14159
4 area = pi * radius ** 2
5 print(area) # Output: 19.6349375
```

1.3 Programs vs Recipes

- Like a **recipe**, a program has:
 - **Ingredients** → data
 - **Steps** → instructions
-

1.4 Real-Life Analogy: Importance of Instruction Order

- Computers **cannot infer context**; instructions must be clear and ordered.
- Everyday actions can also be represented as step-by-step commands.

```
1 # Sequence to join a Zoom class
2 print("1. Turn on the computer")
3 print("2. Open web browser")
4 print("3. Enter the URL: https://canvas.sydney.edu.au")
5 print("4. Click on the Zoom link")
6 print("5. Join the meeting")
```

1.5 Programming Reality

- Computers **only do what they're told**, but they do it very fast and accurately.
 - But they **require exact and unambiguous instructions**.
-

1.6 Understanding Computers

- Computers are logical and require **precise syntax**.

```
1 # Incorrect syntax (will cause an error)
2 # print("Hello"
3
4 # Correct syntax
5 print("Hello") # Output: Hello
```

1.7 Activity: Instructing a Friend to Build Lego

- You must describe each step **clearly and specifically**.

```
1 # Simulated instructions
2 print("Pick up the blue brick")
3 print("Place it on top of the green brick")
4 print("Press it firmly to connect")
```

1.8 Programs = English Instructions + Programming Language

- We need to **translate our natural-language instructions** into code using a programming language.
-

1.9 Compilers vs Interpreters

- **Compiler:**

- Converts the **entire source code** into machine code before execution.
- **Definition:** A compiler is a tool that converts the source code of an entire program into machine code. This process is usually completed before the program is run.
- **Workflow:**
 1. Reading source code: The compiler will first read and analyze the source code of the entire program.
 2. Compilation: Convert the source code into machine language (generate object code).
 3. Output: After compilation is completed, an executable file or object file is generated.
- Example: C, C++
- **Interpreter:**
 - Translates and executes the **code line-by-line** at runtime.
 - **Definition:** An interpreter is a tool that reads and executes source code line by line. It converts the code in real time at runtime.
 - **Workflow**
 1. Read source code: The interpreter reads each line of the source code.
 2. Interpretation and execution: The read code is translated into machine language and executed immediately.
 - Example: Python

```
1 # Python is an interpreted language
2 x = 10
3 print(x * 2) # Output: 20
```

1.10 Core Concepts in Programming

1. **Input** – Getting data from the user or other sources.
2. **Output** – Displaying or saving results.
3. **Processing** – Performing calculations or logic.
4. **Conditionals** – Using `if` statements for decisions.
5. **Loops** – Repeating actions using `while` or `for`.

Example:

```
1 # Input, conditional, and output
2 age = int(input("Enter your age: "))
3 if age >= 18:
4     print("You are an adult")
5 else:
6     print("You are a minor")
```

1.11 What Tools Do You Need to Program?

1. **Tools:**
 - Text editor (e.g., VS Code)
 - Terminal or command line

- IDEs (e.g., PyCharm)

2. Workflow:

- Plan → Code → Test → Debug

1.12 Terminal

1.12.1 Command Line Interface (CLI) Basics

```
1  # Show current working directory
2  pwd
3
4  # List files and directories
5  ls
6
7  # Make a new directory
8  mkdir myfolder
9
10 # Change directory
11 cd myfolder
12
13 # Create a new file
14 touch hello.py
15
16 # Move a file
17 mv hello.py ../
18
19 # Copy a file
20 cp hello.py copy_hello.py
```

These are run in a terminal (Linux/macOS) or Git Bash/WSL (Windows).

1.12.2 Editing Code with VIM

Steps to use VIM:

1. Open a file:

```
1 | vim hello.py
```

1. Press **i** to enter insert mode and start typing.
 2. Press **Esc** to leave insert mode.
 3. Type **:w** to save and **:q** to quit.
-

1.12.3 Writing Your First Python Program

```
1  # File: hello.py
2  print("Hello, World!") # Output: Hello, World!
```

Run it using:

```
1 | python3 hello.py
```

1.12.4 Using the Python Interpreter (REPL)

You can launch Python's interactive shell with:

```
1 | python3
```

In the REPL, you can type Python code line-by-line:

```
1 | >>> print("Welcome to Python")
2 | Welcome to Python
3 | >>> 5 + 3
4 | 8
```

To exit:

- Windows: `Ctrl + Z`
- macOS/Linux: `Ctrl + D`
- Or type `quit()`

You can also use the enhanced interactive shell:

```
1 | ipython
```

1.12.5 The `print()` Function in Python

- **Purpose:**
 - Display information in the terminal.

Examples:

```
1 | # Print a string
2 | print("This is Python!") # Output: This is Python!
3 |
4 | # Print a number
5 | print(123) # Output: 123
6 |
7 | # Print multiple items
8 | print("Sum:", 2 + 3) # Output: Sum: 5
9 |
10 | # Newline character
11 | print("Line1\nLine2") # Output:
12 | # Line1
13 | # Line2
14 |
15 | # Single or double quotes
16 | print('Python is fun') # Output: Python is fun
17 | print("Python is also fun") # Output: Python is also fun
```

Common Error (from the slides):

```
1 # Incorrect – causes a syntax error
2 # print('$ $')
3
4 # Correct:
5 print('$ $') # Output: $ $
```

1.12.6 Compiling Python to Bytecode

Although Python is an interpreted language, you can compile it to `.pyc` bytecode:

```
1 python3 -m py_compile hello.py
```

This creates a compiled version of the script:

```
1 __pycache__/hello.cpython-<version>.pyc
```

2. Variables and Data Types

1. Variables

1.1 What is a Variable

- A variable is a *named container* used to store data so it can be referenced and reused.
- It makes code more readable, reusable, and easier to debug.

```
1 name = "Alice"
2 print(name) # Output: Alice
```

1.2 Variable Characteristics in Python

- Python is a **dynamically-typed language**, so you don't need to declare the type beforehand.
- A variable gets its type automatically and can be reassigned to a different type.

```
1 x = 10 # x is an integer
2 x = "hello" # x is now a string
```

C is a statically typed language. Once a variable of a specific type is declared, it cannot be assigned a different data type.

1.3 Data Types and Objects

- **Data Types**
 - A set of values and a set of operations defined on those values.
 - Operators are representation of a data-type operation. Can be unary, binary and ternary.
 - Primitive types are basic data types that uses fixed amount of memory. Examples of these are usually: int, float, char, bool.
- **Object**

- **Objects are blocks of memory** that contain related data and associated operations for that data.
- **One object can contain many other Objects.** They can have customised operators defined by the programmer i.e., you define the actions of operators.
- **In Python, everything is an Object!**

1.4 Naming Rules for Variables

- **Must start with a letter or an underscore** `_`.
- **Cannot contain spaces or special characters.**
- **Case-sensitive:** `x` and `X` are different.
- **Avoid using reserved keywords** like `for`, `if`, `print`, etc.
- **Use meaningful names**, and follow naming conventions (e.g., `snake_case` for variables, `UPPER_CASE` for constants).
- **All CAPS indicate constants** e.g. `FLAT_RATE`

```
1 student_name = "Bob"
2 _studentAge = 20
3 # 5th_student = "Invalid" # Invalid: starts with a digit
```

Reserve Keywords

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

- Do not need to memorize.
- Most development environments will highlight these keywords in a different colour.

2. Data Types

In Python, **everything is an object**. The basic built-in types include:

2.1 Integer (`int`)

- Represents whole numbers, e.g., `-1`, `0`, `100`.
- Used for counting, indexing, etc.

```
1 age = 25
2 print(age + 5) # Output: 30
```

2.2 Float (float)

- Represents decimal or scientific notation numbers.
- Watch out for floating point precision issues.
- **Floating-point numbers are approximations of the real numbers.** Be careful when performing comparisons with floating-point numbers.
- Accuracy: **Integers are exact but floating-point numbers are not.**
- Range: If you are dealing with **very large or small numbers**, you have to use float - their range is much bigger.

```
1 pi = 3.14159
2 e = 6.02e23 # Scientific notation
3
4 price = 1.5
5 quantity = 3
6 total = price * quantity
7 print(total) # Output: 4.5
```

2.3 Boolean (bool)

- Only two values: `True` and `False`
- Often used in conditions and logical operations.

```
1 is_raining = True
2 have_umbrella = False
3 is_wet = is_raining and not have_umbrella
4 print(is_wet) # Output: True
```

Data Type - bool

- Two possible values – `False` (0) and `True` (1).
- Comparison operators are mixed operators that will result in a Boolean.

	<i>op</i>	<i>meaning</i>	True	False
	<code>==</code>	<i>equal</i>	<code>2 == 2</code>	<code>2 == 3</code>
	<code>!=</code>	<i>not equal</i>	<code>3 != 2</code>	<code>2 != 2</code>
	<code><</code>	<i>less than</i>	<code>2 < 13</code>	<code>2 < 2</code>
	<code><=</code>	<i>less than or equal</i>	<code>2 <= 2</code>	<code>3 <= 2</code>
	<code>></code>	<i>greater than</i>	<code>13 > 2</code>	<code>2 > 13</code>
- Boolean operators	<code>>=</code>	<i>greater than or equal</i>	<code>3 >= 2</code>	<code>2 >= 3</code>

Comparisons with int operands and a bool result

2.4 String (str)

- A sequence of characters enclosed in **single or double** quotes.
- Supports concatenation, repetition, and string methods.

```

1 name = "Alice"
2 greeting = "Hello, " + name
3 print(greeting) # Output: Hello, Alice
4
5 # String methods
6 print(name.upper()) # Output: ALICE
7 print(name.lower()) # Output: alice
8 print(name.capitalize()) # Output: Alice. Returns a string where the first character is upper
  case, and the rest is lower case.
9 print(name.swapcase()) # Output: aLICE. Used to convert all uppercase letters in a string to
  lowercase letters, and all lowercase letters to uppercase letters.

```

- **Backslash character** `\` in string denotes escape character
- `\n` - Newline character
- `\t` - Tab character

```

1 print("line1\nline2")
2 # Output:
3 # line1
4 # line2
5
6 print("backslash: \\")
7 # Output: backslash: \

```

- **String concatenation** using `+` operator

```

1 greeting = "Hello"
2 name = "Alice"
3 message = greeting + " " + name
4 print(message) # Output: Hello Alice

```

- **String repetition** using `*` operator

```

1 print("ha" * 3) # Output: hahaha
2 print("-" * 10) # Output: -----

```

2.5 Type Annotation

- Type annotations in Python are used to specify variable types, but they are **not mandatory**.

```

1 x: int = 10
2 name: str = "Alice"
3 print(x, name) # 输出: 10 Alice

```

2.6 Type Errors

1) Invalid Operations

Arithmetic operations like subtraction (`-`), division (`/`), or addition between a string and a number are not valid and will result in a `TypeError`.

```
1 str_1 = 'Hello'
2 result = str_1 - 'World' # This will raise a TypeError
```

2) Adding Strings and Integers

You cannot directly add a string and an integer.

```
1 age_str = 'I am '
2 age_int = 25
3 result = age_str + age_int # This will raise a TypeError
```

3) Multiplying Strings with Floats or Strings

Multiplying a string by a float or another string is not allowed.

```
1 str_1 = 'Hello'
2 result = str_1 * 2.5 # This will raise a TypeError
3
4 str_1 = 'Hello'
5 result = str_1 * "2" # This will raise a TypeError
```

Important Notes

- The arithmetic operators that will work on strings in Python are: `+` on 2 strings, and `*` on a string and integer.
- String multiplication is a Pythonic feature! Other languages often don't offer this luxury!

3. Name of Different Parts of Statement

3.1 literal, Variable, Operator, Expression

- An expression is a combination of literals, variables, and operators that evaluates to a value.
- **literal:** `5`, `2`
- **Variable:** `length`, `width`, `area`
- **Operator:** `=`, `*`
- **Expression:** `length * width`
 - In the expression `length * width`:
 - `length` and `width` are **operands** because they are what the `*` operator acts on.

```
1 length = 5
2 width = 2
3 area = length * width
4 print(area) # Output: 10
```

3.2 Types of Operators

- Arithmetic: `+`, `-`, `*`, `/`
- Comparison: `==`, `!=`, `<`, `>`, `<=`, `>=`
- Boolean: `and`, `or`, `not`

```

1 print(5 + 2)      # Output: 7
2 print(5 > 2)      # Output: True
3 print(not False)  # Output: True

```

Truth Table

- All possible inputs and output combinations for each operation can be looked up in the truth table:

not



X	Z
0	1
1	0

and



X	Y	Z
0	0	0
0	1	0
1	0	0
1	1	1

or



X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	1

How to read arithmetic operators

- `+` : add
- `-` : subtract
- `/` : division
- `*` : multiply
- `**` : exponentiation (raise to a power)
- `%` : modulus (remainder after a division)
- `//` : floor division (rounds down)

4. Type Casting

4.1 Explicit Casting

- Use built-in functions: `int()`, `float()`, `str()`

```

1 x = "123"
2 y = int(x)
3 print(y + 1) # Output: 124

```

4.2 Implicit Casting

- Python automatically casts types in mixed-type expressions.
- **Example cases** where results are implicitly converted to float:
 - **Division between a float and int**

- Division between two int
- Addition between an int and float

```
1 a = 2
2 b = 3.5
3 c = a + b
4 print(c)          # Output: 5.5
5 print(type(c))    # Output: <class 'float'>
6
7 d = a / b
8 print(type(d))    # Output: <class 'float'>
9
10 f = 4
11 g = f / a
12 print(g)         # Output: 2.0
13 print(type(g))   # Output: <class 'float'>
```

5. Input and Output

5.1 `input()` Function

- Reads a line from standard input (keyboard), **converts it to a string, stripping the newline character**.

```
1 name = input("Enter your name: ")
2 print("Hello, " + name)
```

5.2 Formatted Output

- You can use either f-strings or `.format()` for cleaner output.

```
1 name = "Alice"
2 age = 25
3 print(f"{name} is {age} years old.")          # Output: Alice is 25 years old.
4 print("{} is {} years old.".format(name, age)) # Output: Alice is 25 years old.
```

5.3 Format Specifications

```
1 # Padding
2 quantity = 7
3 print(f"{quantity:03d}") # Output: 007 (padded to 3 digits with zeros)、
4 print(f"{quantity:3d}")  # Output:    7 (padded to 3 digits with spaces)、
5 print(f"{quantity:03}")  # Output: 007 Automatic type inference is integer + left-padded with 0
6
7 num = 3.14
8 print(f'{num:04}')
```

Normal output: 3.14 (not an integer, no zeros added)

print(f'{num:04d}') # Report an error: Unknown format code 'd' for object of type 'float'

```
10
11 # Specify Decimal Places
12 pi = 3.1415926
13 print(f"{pi:.2f}") # Output: 3.14
```

3. Conditionals

3.1 Why Do We Need Decisions?

- In **real life**, we constantly make decisions:
 - If it rains, we use an umbrella.
 - If we are hungry and tired, we order a meal.
- In **computer programs**, decisions allow different actions based on different inputs:
 - Self-driving cars respond differently to red and green lights.
 - Spam filters decide whether to flag an email.

3.2 Control Flow Basics

- Python code is executed **top-to-bottom** by default.
- Conditional statements **change the flow** depending on logic.
- **Flowcharts**

Flowcharts

- Present plan of solution independent of programming language.



An oval indicates beginning or end of a program.

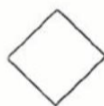


A parallelogram is a point where there is input to or output from the program.



A rectangle indicates the assignment of a value to a variable, constant, or parameter. the assigned value can be the result of a computation. The computation would also be included in the rectangle.

- We will use the



A diamond indicates a point where a decision is made.

Image credit: https://www.owlnet.rice.edu/~ceng303/manuals/fortran/FOR3_3.html

3.3 Simple Branching: `if` Statement

Syntax:

```
1 if condition:
2     statement_block
```

Example:

```

1 x = 10
2 if x > 0:
3     print("x is positive") # Output: x is positive

```

Explanation:

- The block inside `if` executes **only if the condition is `True`**.

Control Flow: if

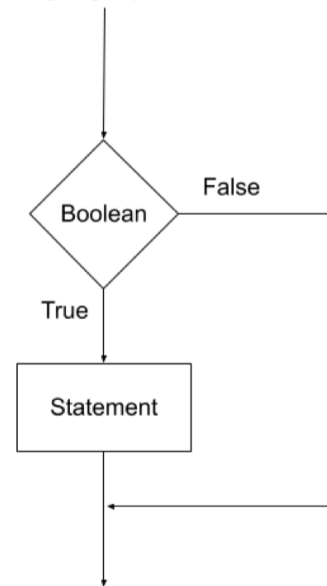
• Syntax:

```

if <Boolean_expression>:
    Statement

```

• Flowchart



3.4 Booleans and Comparison Operators

Boolean values:

- `True` and `False`

Comparison operators:

Operator	Meaning	Example	Result
<code>==</code>	Equals	<code>5 == 5</code>	<code>True</code>
<code>!=</code>	Not equals	<code>5 != 3</code>	<code>True</code>
<code>></code>	Greater than	<code>5 > 3</code>	<code>True</code>
<code><</code>	Less than	<code>5 < 3</code>	<code>False</code>
<code>>=</code>	Greater or equal	<code>5 >= 5</code>	<code>True</code>
<code><=</code>	Less or equal	<code>3 <= 4</code>	<code>True</code>

Example:

```
1 score = 85
2 if score >= 50:
3     print("You passed!") # Output: You passed!
```

3.5 Two-Way Branching: `if-else`

When to use: When you have more than two possibilities and need **more than two branches**.

Operation Logic: Condition is checked in order of appearance. If the first is false, the next is checked and so on. Only the first true branch runs.

Syntax:

```
1 if condition:
2     statement1
3 else:
4     statement2
5
6 # Incorrect usage
7 if condition:
8     doSomething()
9 else doAnotherThing()
```

Example:

```
1 score = 40
2 if score >= 50:
3     print("Pass")
4 else:
5     print("Fail") # Output: Fail
```

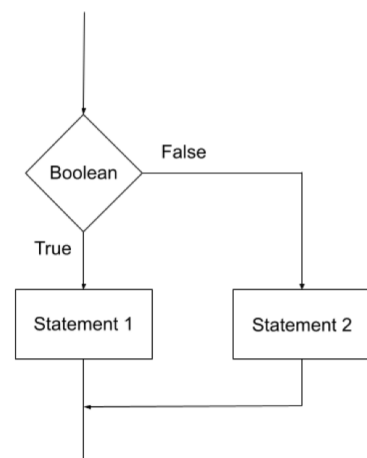
Control Flow: `if... else`

• Syntax:

```
if <Boolean expression>:
    Statement 1
else:
    Statement 2
```

Warning:
DO NOT put expressions after keyword `else`.

• Flowchart



3.6 Multi-Way Branching: `if-elif-else`

Syntax:

```
1 if condition1:
2     block1
3 elif condition2:
4     block2
5 ...
6 else:
7     default_block
```

Example:

```
1 mark = 75
2 if mark >= 85:
3     print("High Distinction")
4 elif mark >= 75:
5     print("Distinction") # Output: Distinction
6 elif mark >= 65:
7     print("Credit")
8 elif mark >= 50:
9     print("Pass")
10 else:
11     print("Fail")
```

Rules:

- Multiple `elif` allowed.
- `else` is optional but **must come last**.
- **Only one block executes** (first true condition).

3.7 Boolean Logic Operators

Operator	Description	Example	Result
<code>and</code>	Logical AND	<code>True and False</code>	<code>False</code>
<code>or</code>	Logical OR	<code>True or False</code>	<code>True</code>
<code>not</code>	Logical NOT	<code>not True</code>	<code>False</code>

Example:

```
1 thirsty = True
2 hungry = False
3 if thirsty and hungry:
4     print("Order meal set")
5 else:
6     print("No meal set") # Output: No meal set
```

If there are no brackets interfering, Python will perform the operations in the following order:

- Not
- And

- Or

I remember it by thinking **NAO** (now!).

3.8 Compound Conditions

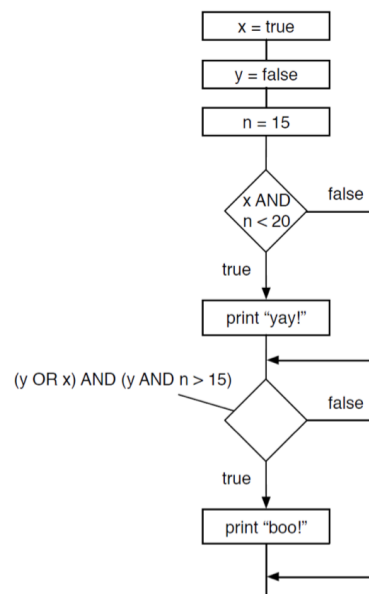
- You can group multiple conditions using parentheses:

```
1 x = 10
2 y = 20
3 if (x > 5 and y > 15) or x == 10:
4     print("Condition met") # Output: Condition met
```

Example: Complex Condition

- What is the output of the program?

Conditions	Result
n<20	
x AND n<20	
y OR x	
y AND n>15	
(y OR x) AND (y AND n>15)	



3.9 Nested if Statements

Syntax:

```
1 if condition1:
2     if condition2:
3         block
```

Example:

```
1 x = 10
2 if x > 0:
3     if x < 20:
4         print("x is between 0 and 20") # Output: x is between 0 and 20
```

Example: Nested if

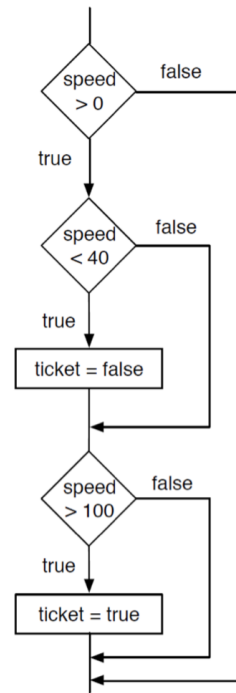
Activity

Predict the value in variable `ticket` at the end of the program if `speed` is initialized to:

- 5
- 40
- 80
- 100

Write a program to check your answer.

- Go to: Ed Lesson



3.10 The `pass` Statement

Used as a placeholder when a block of code is **syntactically required** but nothing needs to be done.

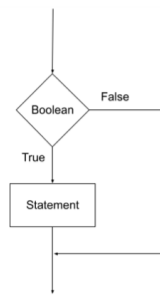
```
1 x = 10
2 if x > 0:
3     pass # Do nothing
```

3.11 Summary

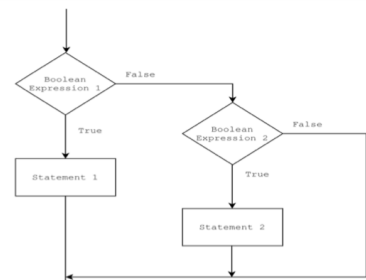
- Conditional statements control execution paths.
- Types:
 - `if`
 - `if-else`
 - `if-elif-else`
 - Nested `if`
- Use Boolean expressions to decide which block runs.

Summary

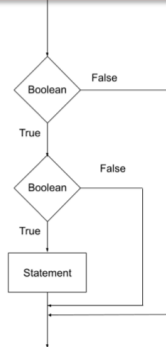
- Selection statements allows the flow of execution to follow a specific branch *depending on the condition*.
- Different types of branching:



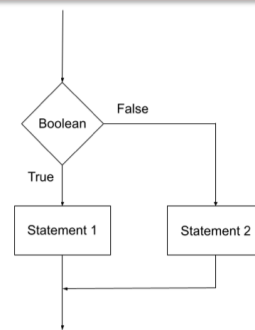
if



if elif



Nested ifs



if-else

Summary

- Selection statements allows the flow of execution to follow a specific branch *depending on the condition*.
- Different types of branching:

```
1 # simple if
2 number = 11
3
4 if number > 10:
5     result = number % 10
6
7 print(result)
```

if

```
1 # simple if elif
2 number = 7
3
4 if number > 10:
5     result = number % 10
6 elif number > 2:
7     result = number % 2
8
9 print(result)
```

if elif

```
1 #simple Nested if
2 number = 7
3
4 if number < 10:
5     if number > 0:
6         result = number * 10
7
8 print(result)
```

Nested ifs

```
1 #simple if-else
2 number = 7
3
4 if number > 10:
5     result = number % 10
6 else:
7     result = number
8
9 print(result)
```

if-else

4. Containers

4.1 List

4.1.1 Concept

1. Definition

- o In Python, a list is an **ordered, mutable (changeable)** data structure that can store any number of elements. These elements can all be of the **same type** or can be of **different types**.

- A list is denoted by square brackets `[...]`, with elements separated by commas.

2. Difference from Arrays

- In many programming languages, **arrays** typically store elements of **a single type**, and their **size is fixed**. In Python, **lists** can store different types of elements and **grow or shrink dynamically**.
- While Python does provide array-like structures (via the `array` module or NumPy arrays), they serve different purposes. In most Python applications, the built-in list is sufficient.

3. Memory Structure

- Under the hood, a **list** is often implemented as **a contiguous block of memory**. Python usually **allocates some extra space** so that appending elements doesn't require frequent memory reallocation.
- For extremely large lists or **extensive insert/delete operations** in the middle of the list, **performance considerations may arise**, and you might need alternative data structures.

4.1.2 Initializing

1. Using Square Brackets Directly

```
1 # Create an empty list
2 empty_list = []
3 print(empty_list) # Output: []
4
5 # Create a list with multiple elements
6 names = ["Alice", "Bob", "Charlie"]
7 print(names) # Output: ['Alice', 'Bob', 'Charlie']
8
9 # A list can contain elements of different types
10 mixed_list = [1, "Hello", 3.14, True]
11 print(mixed_list) # Output: [1, 'Hello', 3.14, True]
```

2. Using the Built-in `list()` Function

```
1 # Converting another iterable object into a list
2 str_to_list = list("ABC")
3 print(str_to_list) # Output: ['A', 'B', 'C']
4
5 # Converting a tuple into a list
6 tuple_data = (1, 2, 3)
7 tuple_to_list = list(tuple_data)
8 print(tuple_to_list) # Output: [1, 2, 3]
```

3. List Comprehension

- Although not specifically covered in detail in the slides, this is a common and concise way to create and process lists in Python.

```
1 squares = [x*x for x in range(5)]
2 print(squares) # Output: [0, 1, 4, 9, 16]
```

4.1.3 Accessing and Indexing

1. Indexing (Zero-based)

- Python uses **zero-based indexing**, meaning the first element is at index `0`.

```
1 fruits = ["Apple", "Banana", "Cherry", "Date"]
2 print(fruits[0]) # Output: Apple
3 print(fruits[2]) # Output: Cherry
```

2. Negative Indexing

- Python supports negative indexing, where `-1` refers to the **last element**, `-2` to the second last, and so on.

```
1 print(fruits[-1]) # Output: Date
2 print(fruits[-2]) # Output: Cherry
```

3. Getting the Length

- Use the built-in `len()` function to get the number of elements in a list.

```
1 print(len(fruits)) # 4
```

4. Index Out of Range

- Accessing an index beyond the list's range raises `IndexError`.

```
1 # print(fruits[4]) # IndexError: list index out of range
```

4.1.4 Mutability

1. Updating Elements

- Lists are mutable, so you can modify elements in place using indexing.

```
1 numbers = [10, 20, 30]
2 numbers[1] = 99
3 print(numbers) # Output: [10, 99, 30]
```

2. Adding Elements

- Use `append()` to **add a new element at the end of the list**, or `insert()` to add an element at a specific position.

```
1 numbers.append(40)
2 print(numbers) # [10, 99, 30, 40]
3
4 numbers.insert(2, 50)
5 print(numbers) # [10, 99, 50, 30, 40]
```

3. Removing Elements

- `pop(index)` removes and returns the element at `index` (if no index is specified, it removes the last element).
- `remove(value)` removes the **first occurrence** of `value`.

```

1 popped_value = numbers.pop() # numbers.pop(-1)
2 print(popped_value) # 40
3 print(numbers) # [10, 99, 50, 30]
4
5 numbers.remove(99)
6 print(numbers) # [10, 50, 30]

```

4. Replacing Slices

- You can also replace, insert, or delete multiple elements at once using slicing.

```

1 letters = ["a", "b", "c", "d", "e"]
2 letters[1:3] = ["x", "y"] # [ ] the slicing does not contain the last index
3 print(letters) # ['a', 'x', 'y', 'd', 'e']

```

4.1.5 Common Methods

Below are some methods mentioned in the slides or frequently used in practice:

1. `append(x)`

- Adds element `x` to the end of the list.

```

1 my_list = [1, 2, 3]
2 my_list.append(4)
3 print(my_list) # [1, 2, 3, 4]

```

2. `count(x)`

- Returns the **number of times** `x` appears in the list.

```

1 my_list = ["a", "b", "a", "c", "a"]
2 print(my_list.count("a")) # 3

```

3. `index(x)`

- Returns the **index of the first occurrence of** `x`.

```

1 print(my_list.index("b")) # 1

```

4. `insert(index, x)`

- Inserts `x` at the specified `index`.

```

1 my_list.insert(2, "d")
2 print(my_list) # ['a', 'b', 'd', 'a', 'c', 'a']

```

5. `pop(index)`

- Removes and returns the element at `index`. If no index is provided, removes and returns the last element.

```

1 removed = my_list.pop(2)
2 print(removed) # d
3 print(my_list) # ['a', 'b', 'a', 'c', 'a']

```

6. `remove(x)`

- Removes the first occurrence of `x` from the list.

```
1 my_list.remove("a")
2 print(my_list) # ['b', 'a', 'c', 'a']
```

7. reverse()

- Reverses the list in place.

```
1 my_list.reverse()
2 print(my_list) # ['a', 'c', 'a', 'b']
```

8. sort()

- Sorts the list in ascending order **in place**. To customize sorting, you can use parameters like `key` or `reverse=True`.

```
1 numeric_list = [3, 1, 4, 2]
2 numeric_list.sort()
3 print(numeric_list) # [1, 2, 3, 4]
```

4.1.6 Slicing

1. Basic Syntax

- `list[start:end]` returns elements from index `start` up to (but not including) `end`.
- `start` defaults to `0`, `end` defaults to the **length of the list**.

```
1 alphabet = ["a", "b", "c", "d", "e", "f"]
2 print(alphabet[2:5]) # ['c', 'd', 'e']
3 print(alphabet[2:]) # ['c', 'd', 'e', 'f']
4 print(alphabet[:3]) # ['a', 'b', 'c']
```

2. Step

- You can **specify a step value** using `list[start:end:step]`.

```
1 print(alphabet[::2]) # ['a', 'c', 'e']
2 print(alphabet[1::2]) # ['b', 'd', 'f']
```

3. Negative Index with Slicing

```
1 print(alphabet[-3:]) # ['d', 'e', 'f']
2 print(alphabet[::-1]) # ['f', 'e', 'd', 'c', 'b', 'a']
```

4.1.7 Two-Dimensional Lists

1. Definition

- Python lists can contain other lists, allowing the creation of **two-dimensional or multi-dimensional structures**. This is useful for tabular data.

```
1 matrix = [  
2     [1, 2, 3],  
3     [4, 5, 6],  
4     [7, 8, 9]  
5 ]
```

2. Accessing Elements

- Access elements using `[row_index][column_index]`.

```
1 print(matrix[1])      # [4, 5, 6]  
2 print(matrix[1][2])  # 6
```

3. Modifying Elements

```
1 matrix[2][1] = 88  
2 print(matrix)      # [[1, 2, 3], [4, 5, 6], [7, 88, 9]]
```

4.2 Tuple

4.2.1 Concept

1. Definition

A tuple is an **ordered** and **immutable** (cannot be modified once created) data structure in Python.

- *Ordered* means the elements have a defined order and can be accessed via their index.
- *Immutable* means that once a tuple is created, you cannot change its elements (no addition, deletion, or modification of existing elements).

2. Syntax

- Tuples use parentheses `( )`, with elements separated by commas.
- If there is **only one element**, a **trailing comma is required to distinguish it** from a regular expression or another data type.

```
1 # A single-element tuple requires a trailing comma  
2 single_tuple = (42,)
```

3. Difference from Lists

- A list is *mutable* (can be modified), uses square brackets `[ ]`, and allows add/remove/modify operations.
- A tuple is *immutable*, uses parentheses `( )`, and once created, its elements cannot be changed.

4.2.2 Initializing

1. Creating Tuples Directly

```

1 # Creating a tuple with multiple elements
2 my_tuple = (10, 20, 30)
3 print(my_tuple) # Output: (10, 20, 30)
4
5 # A tuple can also contain different data types
6 mixed_tuple = ("Hello", 3.14, True)
7 print(mixed_tuple) # Output: ('Hello', 3.14, True)

```

2. Using the `tuple()` Constructor

```

1 # Creating a tuple from a list
2 list_data = [1, 2, 3]
3 tuple_from_list = tuple(list_data)
4 print(tuple_from_list) # Output: (1, 2, 3)
5
6 # Creating a tuple from a string
7 str_data = "ABC"
8 tuple_from_str = tuple(str_data)
9 print(tuple_from_str) # Output: ('A', 'B', 'C')

```

3. Empty Tuple

```

1 empty_tuple = ()
2 print(empty_tuple) # Output: ()

```

4. Single-Element Tuples

```

1 # With a comma, it's a tuple
2 single_tuple = ("Hello",)
3 print(single_tuple) # Output: ('Hello',)
4 print(type(single_tuple)) # Output: <class 'tuple'>
5
6 # Without a comma, it's interpreted as a string
7 not_tuple = ("Hello")
8 print(type(not_tuple)) # Output: <class 'str'>

```

4.2.3 Accessing Tuple

1. Indexing

Tuples use zero-based indexing. The first element has index `0`, the second has index `1`, etc.

```

1 my_tuple = ("python", "java", "c++", "go")
2 print(my_tuple[0]) # Output: python
3 print(my_tuple[1]) # Output: java

```

2. Negative Indexing

Negative indices can be used to access elements starting from the end. `-1` refers to the last element, `-2` refers to the second last, and so on.

```

1 my_tuple = (10, 20, 30, 40)
2 print(my_tuple[-1]) # Output: 40
3 print(my_tuple[-2]) # Output: 30

```

3. Slicing

- Syntax: `tuple[start:end]`, including `start` index but excluding `end` index.
- If `start` is omitted, it defaults to `0`. If `end` is omitted, it defaults to the length of the tuple.

```
1 numbers = (1, 2, 3, 4, 5, 6)
2 # Slicing index 1 up to (but not including) index 3
3 slice_tuple = numbers[1:3]
4 print(slice_tuple) # Output: (2, 3)
5
6 # From the beginning to index 3 (excluded)
7 slice_tuple = numbers[:3]
8 print(slice_tuple) # Output: (1, 2, 3)
9
10 # From index 2 to the end
11 slice_tuple = numbers[2:]
12 print(slice_tuple) # Output: (3, 4, 5, 6)
13
14 # Using negative indices
15 slice_tuple = numbers[-3:]
16 print(slice_tuple) # Output: (4, 5, 6)
```

4.2.4 Immutability

1. Cannot Modify Existing Elements

Attempting to reassign a tuple element results in a `TypeError`.

```
1 my_tuple = (10, 20, 30)
2 # my_tuple[1] = 99 # This will raise: TypeError
```

2. Cannot Add or Remove Elements

- No `append()` or `remove()` operations (as found in lists).
- If you want to add or remove elements, you need to convert the tuple to a list, modify it, and convert it back to a tuple.

```
1 my_tuple = (10, 20, 30)
2 temp_list = list(my_tuple)
3
4 # Perform list operations
5 temp_list.append(40)
6 # Convert back to a tuple
7 my_tuple = tuple(temp_list)
8 print(my_tuple) # Output: (10, 20, 30, 40)
```

4.2.5 Common Methods

1. Length

Get the number of elements in a tuple with `len()`.

```
1 my_tuple = (3, 5, 7)
2 print(len(my_tuple)) # Output: 3
```

2. Membership Check

Use the `in` and `not in` operators to check if a value is in a tuple.

```
1 my_tuple = ("apple", "banana", "cherry")
2 print("banana" in my_tuple)      # Output: True
3 print("orange" not in my_tuple)  # Output: True
```

3. Iterating Through a Tuple

Use a `for` loop to iterate over the elements in a tuple.

```
1 my_tuple = ("python", "java", "c++")
2 for item in my_tuple:
3     print(item)
4 # Output:
5 # python
6 # java
7 # c++
```

4. Concatenating Tuples

Use the `+` operator to combine two tuples into a new tuple.

```
1 t1 = (1, 2, 3)
2 t2 = (4, 5, 6)
3 t3 = t1 + t2
4 print(t3) # Output: (1, 2, 3, 4, 5, 6)
```

5. Repeating Tuples

Use the `*` operator to repeat the elements of a tuple.

```
1 t = (1, 2)
2 t_repeated = t * 3
3 print(t_repeated) # Output: (1, 2, 1, 2, 1, 2)
```

4.2.6 Tuples in Functions

1. Returning Multiple Values

- Tuples are often used to **return multiple values at once from a function**.

```
1 def get_user_info():
2     name = "Alice"
3     age = 25
4     country = "Australia"
5     return name, age, country # Returns a tuple
6
7 user_info = get_user_info()
8 print(user_info)             # Output: ('Alice', 25, 'Australia')
9 print(type(user_info))       # Output: <class 'tuple'>
```

2. Tuple Unpacking

- You can directly **unpack a tuple** into separate variables when receiving return values.

```
1 def get_point():
2     x = 10
3     y = 20
4     return (x, y)
5
6 px, py = get_point() # Unpacking
7 print(px) # Output: 10
8 print(py) # Output: 20
```

4.2.7 Packing and Unpacking

1. Packing

- When **assigning multiple values to a single variable** without explicit parentheses `()`, **Python automatically packs them into a tuple**.

```
1 my_tuple = 10, "hello", True # Interpreted as (10, "hello", True)
2 print(my_tuple)
3 # Output: (10, 'hello', True)
```

2. Unpacking

- Assign elements of a tuple to multiple variables.
- **The number of variables must match the number of elements in the tuple.**

```
1 a, b, c = my_tuple
2 print(a) # Output: 10
3 print(b) # Output: hello
4 print(c) # Output: True
```

4.2.8 Converting Between Tuples and Lists

1. Tuple to List

- Convert a tuple to a list with `list()`, allowing modifications.

```
1 t = (1, 2, 3)
2 lst = list(t)
3 lst.append(4)
4 print(lst) # Output: [1, 2, 3, 4]
```

2. List to Tuple

- Convert a list to a tuple with `tuple()`, making it immutable.

```
1 lst = [1, 2, 3]
2 t = tuple(lst)
3 print(t) # Output: (1, 2, 3)
```

4.3 Dictionary

1. Concept

1. Definition

A dictionary in Python is a container type that stores data in the form of **key-value pairs**. It allows for quick lookups of values by their keys (**hash**). Since Python 3.7, dictionaries are ordered based on insertion order. They are also mutable, meaning you can modify them (add, change, or remove items) even after creation.

2. Characteristics

- **Ordered:** In Python 3.7 and above, dictionaries remember the order in which items are inserted.
- **Mutable:** Unlike tuples, dictionaries can be modified after creation.
- **Unique Keys:** Every key in a dictionary must be **unique**. Reassigning a value to an existing key overwrites the old value.
- **Key Types:** Keys are typically **immutable types** (e.g., strings, numbers, or tuples). Lists and other mutable types cannot be used as dictionary keys.
- **Value Types:** Values can be of **any type**, including lists, dictionaries, and other complex structures.

3. Common Use Cases

- **Fast Lookups:** Dictionaries allow quick access to values by using the corresponding key. (**hash**)
- **Data Organization:** Replacing numeric indexes (as with lists) with descriptive keys can make your code more readable.
- **Configuration Management:** Many applications or scripts store configuration parameters in dictionaries for easy modification and retrieval.

2. Initializing

Python offers multiple ways to create a dictionary, with the two most common being curly braces `{}` and the built-in `dict()` function.

1. Using Curly Braces `{}`

```
1 # Method 1: Directly placing key-value pairs in curly braces
2 hero_info = {
3     "name": "Rostam",
4     "age": 30,
5     "weapon": "Spear"
6 }
7 print(hero_info)
8 # Output: {'name': 'Rostam', 'age': 30, 'weapon': 'Spear'}
```

2. Using the `dict()` Constructor

```
1 # Method 2: Using dict() with keyword arguments
2 hero_info = dict(name="Rostam", age=30, weapon="Spear")
3 print(hero_info)
4 # Output: {'name': 'Rostam', 'age': 30, 'weapon': 'Spear'}
5
6 # Alternatively, passing an iterable of tuples or lists
7 hero_info = dict([("name", "Rostam"), ("age", 30), ("weapon", "Spear")])
8 print(hero_info)
9 # Output: {'name': 'Rostam', 'age': 30, 'weapon': 'Spear'}
```

3. Creating an Empty Dictionary

```
1 empty_dict = {}
2 print(empty_dict)      # Output: {}
3 empty_dict_2 = dict()
4 print(empty_dict_2)    # Output: {}
```

3. Accessing Dictionary

Use the key to access dictionary values.

1. Using Bracket Notation `[...]`

```
1 hero_info = {
2     "name": "Rostam",
3     "age": 30,
4     "weapon": "Spear"
5 }
6
7 print(hero_info["name"]) # Output: Rostam
8 print(hero_info["age"]) # Output: 30
```

- Accessing a non-existent key with bracket notation will raise a `KeyError`.

2. Using the `get()` Method

- The `get()` method returns `None` if the key does not exist, unless you specify a default return value.

```
1 hero_info = {"name": "Rostam", "age": 30}
2 print(hero_info.get("weapon"))      # Output: None (no error)
3 print(hero_info.get("weapon", "Unknown")) # Output: Unknown
```

4. Adding, Modifying, and Deleting

1. Adding Key-Value Pairs

- Simply assign a value to a new key if it does not already exist in the dictionary.

```
1 hero_info = {"name": "Rostam"}
2 hero_info["age"] = 30      # Add a new key 'age'
3 hero_info["weapon"] = "Bow" # Add a new key 'weapon'
4 print(hero_info)
5 # Output: {'name': 'Rostam', 'age': 30, 'weapon': 'Bow'}
```

2. Modifying Existing Key-Value Pairs

- If the key already exists, assigning a new value overwrites the old one.

```
1 hero_info = {"name": "Rostam", "weapon": "Spear"}
2 hero_info["weapon"] = "Bow"
3 print(hero_info)
4 # Output: {'name': 'Rostam', 'weapon': 'Bow'}
```

3. Deleting Key-Value Pairs

- Common methods: `del` statement, `pop()`, and `popitem()`.
-

```

1 hero_info = {"name": "Rostam", "age": 30, "weapon": "Bow"}
2
3 # (1) Using del
4 del hero_info["age"]
5 print(hero_info) # {'name': 'Rostam', 'weapon': 'Bow'}
6
7 # (2) Using pop(key)
8 weapon_value = hero_info.pop("weapon")
9 print(weapon_value) # 'Bow'
10 print(hero_info) # {'name': 'Rostam'}
11
12 # (3) Using popitem()
13 # popitem() removes and returns the last inserted key-value pair (Python 3.7+)
14 hero_info["level"] = 10
15 last_item = hero_info.popitem()
16 print(last_item) # ('level', 10)
17 print(hero_info) # {'name': 'Rostam'}

```

4. Clearing a Dictionary

- o Use `clear()` to remove all items at once:

```

1 hero_info = {"name": "Rostam", "age": 30}
2 hero_info.clear()
3 print(hero_info) # Output: {}

```

5. Iterating

You can iterate through a dictionary in three primary ways: by keys, by values, or by key-value pairs.

1. Iterating Over Keys

```

1 hero_info = {"name": "Rostam", "age": 30, "weapon": "Bow"}
2 for key in hero_info.keys():
3     print(key, hero_info[key])
4 # Output:
5 # name Rostam
6 # age 30
7 # weapon Bow

```

2. Iterating Over Values

```

1 for value in hero_info.values():
2     print(value)
3 # Output:
4 # Rostam
5 # 30
6 # Bow

```

3. Iterating Over Key-Value Pairs

```
1 for key, value in hero_info.items():
2     print(key, ":", value)
3 # Output:
4 # name : Rostam
5 # age : 30
6 # weapon : Bow
```

6. Other Common Methods

1. Checking If a Key Exists

```
1 hero_info = {"name": "Rostam", "age": 30}
2 print("name" in hero_info) # True
3 print("weapon" in hero_info) # False
```

2. Getting the Number of Items

```
1 print(len(hero_info)) # 2
```

3. Copying a Dictionary

```
1 # Method 1: Using the copy() method
2 copy_dict = hero_info.copy()
3
4 # Method 2: Using the dict() constructor
5 copy_dict_2 = dict(hero_info)
```

- Note: This creates a **shallow copy**. If the dictionary's values include mutable objects (e.g., lists or nested dictionaries), use a **deep copy** if you need independent objects.

7. Comparison With Other Data Structures

1. Lists

- Lists use integer indices to access elements.
- Dictionaries use keys, which can be strings, numbers, or other immutable types.

2. Tuples

- Tuples are immutable sequences of items.
- Dictionaries are mutable and allow fast insertion, deletion, and lookup by key.

3. Strings

- Strings are immutable sequences of characters.
- Dictionaries store key-value pairs with more complex functionality for data organization.

4.4 String

1. Concept

1. A string is an ordered sequence of characters

- In Python, a string can be seen as a sequence of one or more characters.
- You can define strings using single quotes `'...'`, double quotes `"..."`, or triple quotes `'''...'''` / `"""..."""`.

```

1 # Using single quotes
2 s1 = 'Hello'
3 # Using double quotes
4 s2 = "World"
5 # Using triple quotes (can span multiple lines)
6 s3 = """This is a
7 multiline string"""

```

2. Immutability

- In Python, strings are **immutable** objects. Once created, you cannot directly modify any character by index.
- Attempting to do something like `string_variable[index] = 'new_char'` will raise a `TypeError`.
- If you want to change string content, you usually create a new string via concatenation, slicing, etc.

```

1 my_str = "Hello"
2 # my_str[0] = 'h' # This line will cause an error due to immutability

```

2. Common Operations

1. Indexing

- Like lists and tuples, strings use **0-based indexing**.
- You can use `[index]` to get the character at that position.

```

1 text = "Python"
2 first_char = text[0]
3 last_char = text[-1]
4 print(first_char) # Output: P
5 print(last_char) # Output: n

```

2. Slicing

- Strings, like lists, support slicing in the form `string[start:end]`. It includes the character at `start`, but excludes the character at `end`.
- You can use negative indices, or omit the start or end positions for flexible usage.

```

1 text = "ABCDEFGHI"
2 part1 = text[2:5]
3 part2 = text[:4] # from the beginning to index 3
4 part3 = text[3:] # from index 3 to the end
5 part4 = text[-3:] # last 3 characters
6
7 print(part1) # Output: CDE
8 print(part2) # Output: ABCD
9 print(part3) # Output: DEFGHI
10 print(part4) # Output: GHI

```

3. Length of a string

- Use `len()` to get the number of characters in a string.

```
1 text = "Hello World"
2 length = len(text)
3 print(length) # Output: 11
```

4. Concatenation and Repetition

- The plus sign `+` is used to concatenate strings.
- The asterisk `*` is used to repeat a string multiple times, creating a new string.

```
1 str1 = "Hello"
2 str2 = "Python"
3 result = str1 + " " + str2
4 print(result) # Output: Hello Python
5
6 repeated = str1 * 3
7 print(repeated) # Output: HelloHelloHello
```

5. Membership operator

- Use `in` or `not in` to check if a substring exists in a given string.

```
1 text = "Hello Python"
2 print("P" in text) # Output: True
3 print("python" in text) # Output: False (case-sensitive)
```

3. Common Built-in Methods

Python provides many built-in methods for strings, including:

1. `strip()` / `lstrip()` / `rstrip()`

- These methods remove whitespace characters (like spaces, newlines, etc.) from both ends, left side, or right side of a string, respectively.

```
1 text = " Hello "
2 print(text.strip()) # Output: Hello
3 print(text.lstrip()) # Output: Hello
4 print(text.rstrip()) # Output: Hello
```

2. `lower()` / `upper()`

- Convert all characters of the string into lowercase or uppercase.

```
1 text = "Hello World"
2 print(text.lower()) # Output: hello world
3 print(text.upper()) # Output: HELLO WORLD
```

3. `split()`

- Splits a string into a list based on a specified separator. By default, it splits on whitespace (spaces, newlines, tabs).

```
1 text = "apple,banana,orange"
2 items = text.split(",")
3 print(items) # Output: ['apple', 'banana', 'orange']
```

4. `join()`

- Inserts the specified string between elements of a list (or any iterable) and returns the combined string.

```
1 fruits = ["apple", "banana", "orange"]
2 result = ", ".join(fruits)
3 print(result) # Output: apple, banana, orange
```

5. `find()` / `index()`

- `find()` returns the starting index of the **first occurrence of a substring**. It **returns -1 if the substring does not exist**.
- `index()` works similarly but raises `ValueError` if the substring is not found.

```
1 text = "Hello Python"
2 idx_find = text.find("Python")
3 idx_index = text.index("Python")
4 print(idx_find) # Output: 6
5 print(idx_index) # Output: 6
```

6. `replace()`

- Replaces a specified substring with another and returns a new string. It does not modify the original string (due to immutability).

```
1 text = "Hello Python"
2 new_text = text.replace("Python", "World")
3 print(new_text) # Output: Hello World
4 print(text) # Output: Hello Python (original string unchanged)
```

4. Conversions Between Strings and Other Data Types

1. Converting a string to a list

- Use the `list()` function to break each character of the string into separate elements in a list.

```
1 text = "Hello"
2 text_list = list(text)
3 print(text_list) # Output: ['H', 'e', 'l', 'l', 'o']
```

2. Converting a string to a tuple

- Likewise, use the `tuple()` function to get a tuple of characters.

```
1 text = "Hi!"
2 text_tuple = tuple(text)
3 print(text_tuple) # Output: ('H', 'i', '!')
```

3. Converting between string and numeric types

- If the string content is numeric (e.g., `"123"`), you can convert it to integers or floats using `int()` or `float()`.
- Conversely, convert other types to strings using `str()`.

```

1 num_str = "123"
2 num_int = int(num_str)
3 num_float = float(num_str)
4 print(num_int)      # Output: 123
5 print(num_float)    # Output: 123.0
6
7 # Converting other types to string
8 original_num = 456
9 converted_str = str(original_num)
10 print(converted_str)      # Output: "456"
11 print(type(converted_str)) # Output: <class 'str'>

```

5. Immutability

1. Why immutability?

- The immutable design can simplify certain aspects of string handling, such as thread safety (no data race when multiple threads access the same string).
- When you need to modify a string, you typically build a new string rather than change the old one in place.

2. Example: Creating a new string via concatenation

```

1 text = "Hello"
2 # Want to add "!" at the end
3 new_text = text + "!"
4 print(new_text)  # Output: Hello!
5 print(text)      # Output: Hello (the original string remains unchanged)

```

4.5 Mutable & Immutable

4.5.1 Concept

1. Mutable

- A mutable object can be **modified in-place** after it is created.
- In Python, common mutable objects include: **lists (list)**, **dictionaries (dict)**, and **sets (set)**.
- When you modify a mutable object, Python **does not create a new object**; instead, the change happens directly within the existing object.

2. Immutable

- An immutable object **cannot be modified in-place** after it is created.
- In Python, common immutable objects include: **strings (str)**, **tuples (tuple)**, and **numeric types (int, float, etc.)**.
- Any operation that appears to change an immutable object **actually creates a new object** and reassigns the reference to that new object, rather than altering the original one.

4.5.2 Mutable: List

1. Demonstration That Lists Are Mutable

- A Python list can be changed by indexing, or by using built-in methods such as `append()`, `remove()`, etc.
- These operations do not create a new list object; they modify the original list in-place.

```

1  # Creating a list
2  my_list = [10, 20, 30]
3  print(my_list)
4  # Output: [10, 20, 30]
5
6  # Modifying an element by index
7  my_list[1] = 99
8  print(my_list)
9  # Output: [10, 99, 30]
10
11 # Using append() to add a new element at the end
12 my_list.append(50)
13 print(my_list)
14 # Output: [10, 99, 30, 50]

```

In these examples, each operation modifies the same `my_list` in-place rather than creating a new object.

2. Checking Memory Addresses of a List

- You can use the built-in `id()` function to check an object's memory address.
- Since lists are mutable, performing multiple modifications usually does not change the `id` of the list.

```

1  my_list = [1, 2, 3]
2  print(id(my_list))
3  # Example output: 139766731625224 (varies by system)
4
5  my_list.append(4)
6  print(id(my_list))
7  # Example output: 139766731625224 (remains the same, indicating in-place modification)

```

4.5.3 Immutable: String

1. Demonstration That Strings Are Immutable

- Strings in Python are immutable. Once created, you cannot modify them in-place.
- Attempting to change a character at a specific index will result in an error.

```

1  my_str = "Hello"
2  # Trying to assign a new character to a specific index (this will cause an error)
3  # my_str[1] = "a" # TypeError: 'str' object does not support item assignment

```

2. Reassigning Strings and Memory Addresses

- When you perform operations such as concatenation (`+=`) on a string, Python actually creates a **new** string object rather than modifying the existing one.
- Using `id()` again, we can see how each operation leads to a different object in memory.

```

1  my_str = "Hello"
2  print(my_str, id(my_str))
3  # Output example: Hello 139766731650416
4
5  # Concatenation creates a new string object
6  my_str = my_str + " World"
7  print(my_str, id(my_str))
8  # Output example: Hello World 139766731653136

```

The two `id()` values differ, showing that a new string was created rather than modifying the original.

4.5.4 Another Immutable Example: Tuple

- Like strings, tuples are also immutable. After creation, you cannot modify any element in-place.
- You can read or iterate through a tuple's elements, but attempts to assign to a specific index will fail.

```
1 my_tuple = (1, 2, 3)
2 print(my_tuple)
3 # Output: (1, 2, 3)
4
5 # Attempting to modify a tuple's element (this will cause an error)
6 # my_tuple[0] = 10 # TypeError: 'tuple' object does not support item assignment
```

4.6 Command Line Arguments

1. Concept

Command line arguments refer to additional information passed to a program when it is executed from the command line. Usually, you might use `input()` in a Python script to interact with users, but in certain scenarios (such as automated scripts, batch operations, or calling a Python program from a shell environment), command line arguments can be more convenient and flexible.

- **Typical Usage Scenarios**
 1. **Batch processing tasks:** For example, running a Python script on a schedule, needing to pass different file names or configurations each time.
 2. **Script tools:** Writing a small utility script and specifying different modes or input files via command line.
 3. **Automated testing:** In automated test scripts, parameters can be passed for different test cases, making multiple tests quick and straightforward.

2. Obtain Command Line Arguments

- **The `sys.argv` list**

In Python, command line arguments are stored in the list `argv` under the `sys` module. To use it, you must import the `sys` module, for example:

```
1 import sys
2
3 print(sys.argv) # This will print out the list of command line arguments.
4 # If we run: python my_script.py hello world
5 # The output might look like:
6 # ['my_script.py', 'hello', 'world']
```

- `sys.argv[0]` usually contains the script name (e.g., `my_script.py`).
- `sys.argv[1]` and subsequent elements contain the actual parameters passed to the script (e.g., `'hello'`, `'world'`).

- **Index Access**

Since `sys.argv` is a list, you can access or slice it as you would any other list.

```
1 import sys
```

```

2
3 script_name = sys.argv[0]
4 arg1 = sys.argv[1]
5 arg2 = sys.argv[2]
6
7 print(script_name) # Print the script name
8 # If the command was: python my_script.py hello world
9 # Output:
10 # my_script.py
11
12 print(arg1) # Print 'hello'
13 # hello
14
15 print(arg2) # Print 'world'
16 # world

```

Note: If not enough arguments are provided when running the script, trying to access a missing index will cause an `IndexError`.

3. Pass Command Line Arguments

1. Running a Python script from the operating system's terminal (or command prompt)

The general format is:

```
1 python script_name.py arg1 arg2 arg3 ...
```

For example:

```
1 python my_script.py hello world 123
```

- `my_script.py`: the script file name
- `hello`, `world`, `123`: three command line arguments passed to the script

2. Result Explanation

- `sys.argv[0]` = `'my_script.py'`
- `sys.argv[1]` = `'hello'`
- `sys.argv[2]` = `'world'`
- `sys.argv[3]` = `'123'`

5. Loops

5.1 Why Loops

1. Repetitive scenarios in real life

- Many daily operations involve repetitive behaviors:
 - Make a cup of tea: put tea leaves into the cup, pour hot water, stir, and repeat several times if multiple cups are needed.
 - Shopping: Choose gifts for everyone until everyone has a gift.
 - Learn programming: Keep trying exercises until you fully master them or give up completely.
- These examples all illustrate that in life we often encounter the need to "execute certain steps multiple

times".

2. Repetitive requirements in programming

- In programming, if there is no loop, you have to "copy and paste" the same code every time you encounter repetitive work. This is not only cumbersome, but also prone to errors.
- Using a loop structure, you can tell the computer "when a certain condition is met, repeatedly execute a piece of logic", thereby improving the maintainability, readability and scalability of the code.

5.2 Classification of Loops

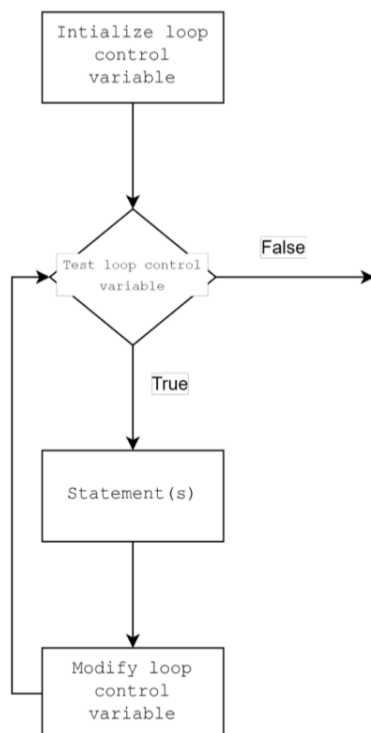
1. Is the number of loops known in advance

- Count control: Yes, we usually know how many times to repeat.
- Event control: No, it stops only when a certain event or condition occurs, and may be executed 0, 1, or n times.

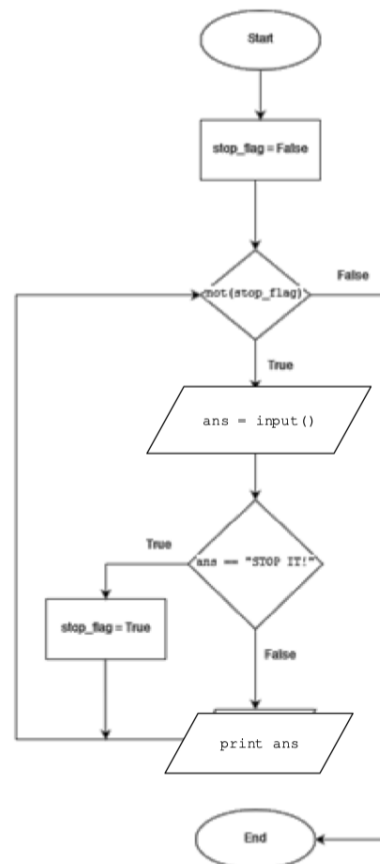
2. Typical implementation method

- Count control:
 - `for i in range(...)` statement is very common;
 - Or use methods such as `while i <= N` (or `while count < N`).
- Event control:
 - `while <some_flag>:` Detect events in the loop body;
 - Modify the loop variable or directly `break` when encountering a "sentinel value".

count-controlled loop



event-controlled loop



3. Application scenario

- Count control: Process a list of fixed length, do fixed rounds of statistics, summation, or repeat a fixed number of operations, etc.

- Event control: Wait for the user to enter a specific command to exit, or read a file until there is no new content, or let the program end under a certain condition.

5.3 Control Flow: While

1. Basic Concept

- **Definition**

A `while` loop is one of the fundamental looping constructs. It repeats its body of code as long as a given boolean expression (condition) remains `True`. Once the condition becomes `False`, execution exits the loop.

- **Use Cases**

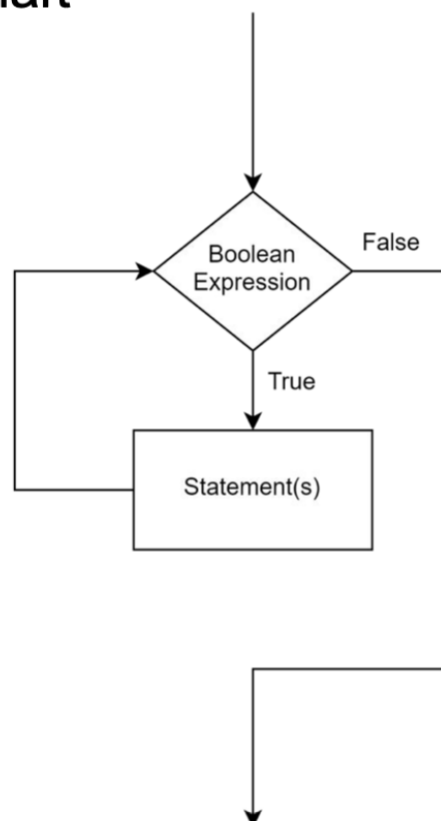
- **Repeated execution until a stop condition is met.** For example, letting the user input data until a special value is entered.
 - Especially useful when the number of iterations is **unknown** beforehand, i.e., in **event-controlled** scenarios.

2. Flow Overview

A simplified flowchart for a `while` loop:

```
1  1. Initialize loop variable
2  2. Check loop condition
3      ┌ If True ─┐
4      │ Execute  │
5      │ loop body │
6      │ Update var │
7      └─────────> Go back to step 2
8  If False -> Exit loop -> Continue with the rest of the program
```

- Flowchart



3. Common Pitfalls and Solutions

1. Infinite Loop

- Occurs if the condition never becomes `False` or the loop variable is never updated.
- Example:

```
1 x = 0
2 while x < 5:
3     print("Hello") # Will print forever because x is never updated
```

- **Solution:** Update `x` inside the loop, such as `x += 1`.

2. Condition Always False

- If the condition starts off as `False`, the loop body never executes.
- Example:

```
1 x = 6
2 while x < 5:
3     print("Will not run")
```

- Since `6 < 5` is `False`, the loop never starts.

3. Incorrect Variable Updates

- Mistakes in the increment, decrement, or logic can yield incorrect loop counts or premature termination.
- Double-check your updates to ensure they align with the desired iteration count.

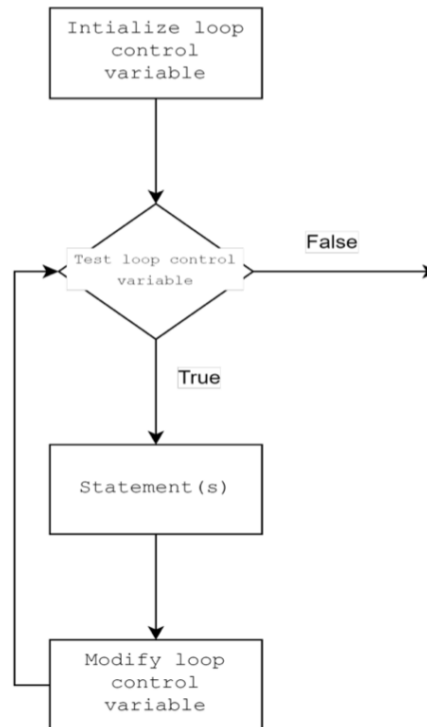
4. Count-Controlled `while` Loop

Scenario: Print numbers from 1 to 5.

```
1 count = 1
2 while count <= 5:
3     print(count) # Output: 1, 2, 3, 4, 5
4     count += 1
```

- **Initialization:** `count = 1`
- **Condition:** `count <= 5`
- **Update:** `count += 1`, which moves `count` toward 6, causing the loop to stop when `count` is 6.

- Flowchart:



5. Event-Controlled `while` Loop (Sentinel Value)

When the number of iterations is **unknown** in advance, an event such as “receiving a special input” may trigger loop termination. Suppose we want the user to keep typing until they enter `"quit"`.

```
1 user_input = "" # Initialize as an empty string
2 while user_input != "quit":
3     user_input = input("Enter something (type 'quit' to stop): ")
4     print("You entered:", user_input) # Output: You entered: ...
5 # When the user eventually types "quit", the loop exits.
```

- **Loop condition:** `user_input != "quit"`.
- As long as this is true, the loop continues. Once the user enters `"quit"`, the condition becomes `False`, and the loop ends.

6. Event-Controlled `while` Loop (Flag)

Sometimes, a **boolean flag** is used to control loop exit. In a number-guessing game, once the user guesses correctly, we set the flag to `True` to exit.

```
1 import random
2
3 secret_number = random.randint(1, 5) # Random integer between 1 and 5
4 found = False # Flag to track if the secret number is found
5
6 while not found:
7     guess = int(input("Guess a number from 1 to 5: "))
8     if guess == secret_number:
9         print("Congratulations, you guessed it!") # Output: Congratulations, you guessed it!
10        found = True
11    else:
```

```
12     print("Nope, try again!") # Output: Nope, try again!
13
14 print("Game Over!") # Output: Game Over!
```

- **Loop condition:** `while not found`.
- Once the user guesses the secret number, `found` changes to `True`, making `while not found` false, thus ending the loop.

5.4 Control Flow: For

1. What Is the `for` Loop

- **Concept:**

In programming, a `for` loop is a control structure used to iterate over a sequence (or any other iterable object). It allows you to perform the same operation on each element of the sequence without writing repetitive code manually.

- **Features:**

1. Suited for scenarios where you **already know** the collection of elements or the number of iterations.
2. Less prone to errors compared to a `while` loop because you do not need to manually manage the loop variable's update.
3. In Python, the `for` loop is commonly used with built-in functions like `range()` for count-based iteration.

2. Basic Syntax

```
1 for variable in iterable_object:
2     # Code block (loop body) to process each element
```

- **variable (loop variable):** Receives each element from the iterable. During each iteration, the loop variable automatically updates to the next element.
- **iterable_object:** Can be a list, tuple, string, dictionary, set, or any custom type that yields one element at a time.
- **loop body:** The code that handles each element, which might include printing, computations, condition checks, etc.

3. Iterating Over a List

Below is a simple scenario: iterating through a list and printing its elements.

```
1 fruits = ["apple", "banana", "cherry"]
2
3 for fruit in fruits:
4     print(fruit)
5 # Output:
6 # apple
7 # banana
8 # cherry
```

- **Explanation:**

1. The list `fruits` contains three elements: `"apple"`, `"banana"`, and `"cherry"`.
2. `for fruit in fruits:` means that each element of the list is assigned to the variable `fruit` during each iteration.
3. The loop body `print(fruit)` prints the element in each iteration.
4. The loop runs three times in total, printing "apple", "banana", and "cherry" in sequence.

4. Iterating Over a String

In Python, strings can be viewed as sequences of characters. Therefore, you can use a `for` loop to access each character in turn.

```
1 text = "Hello"
2
3 for ch in text:
4     print(ch)
5 # Output:
6 # H
7 # e
8 # l
9 # l
10 # o
```

- **Explanation:**

1. The string `"Hello"` is treated as a sequence of five characters.
2. `for ch in text:` assigns each character ('H', 'e', 'l', 'l', 'o') to `ch` in each iteration.
3. The loop body prints each character, resulting in one character per line.

5. Using `range()` With a `for` Loop

Often, you need to repeat something a fixed number of times or use an index-based loop. Python provides a built-in `range()` function, commonly used in `for` loops to generate a series of integers.

- **Basic Example**

```
1 # range(5) generates 0, 1, 2, 3, 4
2 for i in range(5):
3     print("Loop count:", i)
4 # Output:
5 # Loop count: 0
6 # Loop count: 1
7 # Loop count: 2
8 # Loop count: 2
9 # Loop count: 3
10 # Loop count: 4
```

Explanation:

1. `range(5)` produces the integers 0 through 4.
2. The variable `i` takes the values 0, 1, 2, 3, and 4 in each iteration.
3. The loop body is executed 5 times, printing the loop counter each time.

- ****Common Forms of `range()`****

- `range(n)`: generates the sequence `[0, 1, 2, ..., n-1]`.
- `range(start, end)`: generates integers from `start` to `end-1`.
- `range(start, end, step)`: generates integers from `start` to `end-1` in increments of `step`. For example, `range(1, 10, 2)` generates `[1, 3, 5, 7, 9]`.

Examples:

```
1 for i in range(2, 7):
2     print(i)
```

```

3  # Output:
4  # 2
5  # 3
6  # 4
7  # 5
8  # 6
9
10 for i in range(1, 10, 3):
11     print(i)
12 # Output:
13 # 1
14 # 4
15 # 7

```

5.5 Infinite Loops

5.5.1 Definition

- An infinite loop refers to a situation where a **loop never terminates on its own** because the exit condition is never met.
- In Python, if you fail to properly update the loop variable or if the loop's condition always remains true, the loop will continue indefinitely until forcibly stopped (e.g., pressing `Ctrl + C` in the console).

5.5.2 Common Examples

1. **Example 1:** `while True` with no break

```

1  # Example 1: This will continuously print "Hello, World!"
2  # and never terminate on its own.
3  while True:
4      print("Hello, World!") # Output: Hello, World!
5      # The program will remain here in a perpetual loop

```

- **Analysis**
 - The condition is permanently `True`, and there is no `break` or other exit strategy. It will keep printing `Hello, World!`.

2. **Example 2:** Loop variable never updated

```

1  # Example 2: Accidental infinite loop
2  i = 0
3  while i < 3:
4      print(i) # Output: 0
5              # Output: 0
6              # Output: 0
7              # ...
8      # There's no update to i inside the loop,
9      # so the condition i < 3 remains true forever.

```

- **Analysis**
 - Since `i` is not updated inside the loop, `i` stays at 0, and `i < 3` is always true, creating an infinite loop.

3. **Example 3:** Event-controlled loop with incorrect logic

```

1 # Example 3: The program is supposed to end when the user types "exit",
2 # but an incorrect condition can lead to an infinite loop.
3
4 user_input = ""
5 while user_input != "exit":
6     user_input = input("Type 'exit' to quit: ")
7     print("You entered:", user_input)
8     # If we accidentally wrote user_input != "quit"
9     # instead of user_input != "exit",
10    # the loop would never exit when the user types "exit".

```

- **Analysis**

- If the sentinel value or condition is incorrectly written, the loop can't break out because the expected exit condition is never detected.

5.5.3 How to Break or Avoid Infinite Loops

1. Manual Interruption

- When running a Python script in a terminal, press **Ctrl + C** to send a keyboard interrupt and stop the program.
- In IDEs like VS Code or PyCharm, you can often click a "Stop" button to terminate.

2. Use `break` Statements

```

1 while True:
2     user_input = input("Enter 'stop' to end the loop: ")
3     if user_input == 'stop':
4         break # Exit loop when user_input is 'stop'
5     print("You entered:", user_input)

```

- This structure allows a `while True` loop to exit once a specific condition is met.

3. Correctly Update Your Loop Variable

```

1 i = 0
2 while i < 3:
3     print("Current value of i:", i)
4     i += 1 # Ensure i is updated each iteration

```

- Always ensure that the loop variable will eventually cause the loop condition to become false.

6. Flow and Functions

6.1 What is a Function?

A function is a **reusable block of code** designed to perform a specific task. It helps break down complex problems into smaller, manageable parts.

6.2 Function Definition Syntax

Use `def` to define a function:

```
1 def greet():
2     print("Hello, world!") # Output: Hello, world!
```

- `def`: Keyword used to define the function
- `greet`: Name of the function (snake_case)
- `()`: May include parameters
- `:`: Indicates the start of the function body
- Indented lines form the function body

Calling the Function:

```
1 greet() # Output: Hello, world!
```

6.3 Function Parameters and Return Values

Function with Parameters:

```
1 def greet(name):
2     print(f"Hello, {name}!") # Output: Hello, Alice!
3
4 greet("Alice")
```

Using `return`:

```
1 def add(x, y):
2     return x + y
3
4 result = add(3, 5)
5 print(result) # Output: 8
```

No `return` defaults to `None`:

```
1 def no_return():
2     print("No return here")
3
4 x = no_return()
5 print(x) # Output: No return here
6         # Output: None
```

6.4 Parameters vs Arguments

- **Parameter**: The variable name inside the function definition
- **Argument**: The actual value you pass to the function during the call

```
1 def multiply(a, b): # Parameters
2     return a * b
3
4 result = multiply(4, 5) # Arguments
5 print(result) # Output: 20
```

6.5 Returning Multiple Values (using tuples)

```
1 def get_info():
2     return "John", 25    # The return format is ("John", 25)
3
4 name, age = get_info()
5 print(name)    # Output: John
6 print(age)     # Output: 25
```

6.6 Why Use Functions?

- **Code Reuse:** Define once, use many times
 - **Decomposition:** Break big tasks into smaller ones
 - **Maintainability:** Easier to test, debug, and maintain
 - **Consistency:** Less duplication, fewer errors
-

6.7 Type Hinting

```
1 def square(x: int) -> int:
2     return x * x
3
4 print(square(4))    # Output: 16
```

Note:

- `x: int` indicates `x` is expected to be an integer
 - `-> int` means the return value is an integer
 - **Python does not enforce types at runtime—this is for clarity and tooling**
-

6.8 Docstrings

```
1 def greet(name):
2     """
3     A function to greet a person.
4
5     Parameters:
6     name (str): Name of the person to greet
7
8     Returns:
9     None
10    """
11    print(f"Hello, {name}!")
12
13 greet("Tom")    # Output: Hello, Tom!
```

Use `help(greet)` to read the docstring.

```
1 help(greet)
```

Output results:

```
1 Help on function greet in module __main__:
2
3 greet(name)
4     A function to greet a person by name.
5
6     Parameters:
7         name (str): The name of the person to greet.
8
9     Returns:
10         None
```

6.9 Variable Scope

Local Variables:

```
1 def example():
2     x = 10
3     print(x)
4
5 example() # Output: 10
6 # print(x) # Error: x is not defined
```

Global Variables:

```
1 x = 5
2
3 def show():
4     print(x) # Output: 5
5
6 show()
```

Modifying Global Variables:

```
1 count = 0
2
3 def increment():
4     global count
5     count += 1
6
7 increment()
8 print(count) # Output: 1
```

6.10 Pure vs Impure Functions

Pure Function:

```
1 def pure_add(a, b):
2     return a + b
```

Impure Function (side effect) have either one or both of the following characteristics:

- **Side effects:** Modify the global state of a program, making the function introduce hidden dependencies within the rest of the program. It is no longer a pure, isolated code block as its coupled with changing things outside its scope.
- **Non-deterministic:** For the same inputs (parameters), it may not always lead to the same output (return value). This is due to the function being influenced by **external factors** (reading in global variables that can change throughout the program), making the function dependent on external states of the program.

```
1 total = 0
2
3 def impure_add(x):
4     global total
5     total += x
6     return total
```

A **side effect** occurs when a **function modifies some state outside of its scope**, such as:

- **Changing a global variable** (we saw this in the previous slide).
- **Modifying a mutable argument** (list and dictionaries).
- `print()` and `input()` (as your terminal's input/output is considered a **global resource**).
 - so `print()` / `input()` **within a function** is considered a side effect.

6.11 Modules

Using built-in modules:

```
1 import math
2 print(math.sqrt(16)) # Output: 4.0
```

Creating custom module (e.g. `mymath.py`):

```
1 def add(x, y):
2     return x + y
```

Main file:

```
1 import mymath
2 print(mymath.add(3, 4)) # Output: 7
```

6.12 Flow of Execution

- Python executes code **top to bottom**, line by line
- Function body only executes when **called**
- Supports call nesting and recursion

6.13 The `main()` Function and `__name__`

```

1 def main():
2     print("Start of program")
3
4 if __name__ == "__main__":
5     main() # Output: Start of program

```

Explanation:

- When the file is run directly, `__name__ == "__main__"`
- When imported, `__name__` becomes the module name
 - `my_module.py` is imported, so its `__name__` is the module name `"my_module"`, i.e. the file name (without the `.py`)

6.14 Debugging with `pdb` (Python Debugger)

Start debugger:

```

1 python3 -m pdb myscript.py

```

Part	Meaning
<code>python3</code>	Execute the program using the Python 3 interpreter.
<code>-m</code>	Indicates running a module instead of running the script file directly.
<code>pdb</code>	Specifies that the module to run is <code>pdb</code> (Python Debugger). It is a module in the Python standard library used to debug Python programs.
<code>myscript.py</code>	The name of the Python script to debug. You can replace it with any file name of your own script.

Manually set breakpoint:

```

1 import pdb
2 pdb.set_trace() # Set a breakpoint, the program will pause here and enter debug mode

```

Python 3.7+:

```

1 breakpoint() # Equivalent to import pdb; pdb.set_trace()

```

Common commands:

- `n`: next line
- `s`: step into
- `p var`: print variable
- `l`: list source code

6.15 Informal Trace (Manual Variable Tracking)

```

1 a = 1
2 b = 2
3 c = a + b
4 print(c) # Output: 3

```

Tracking table:

Step	a	b	c
1	1		
2	1	2	
3	1	2	3

7 Classes and Objects

7.1 Everything Is an Object in Python

- Python is an object-oriented programming language.
- Everything, including strings, numbers, and lists, is an object.
- Objects have **attributes** (data) and **methods** (functions).

```

1 s = "Hello World"
2 print(s.upper()) # Output: HELLO WORLD

```

Explanation:

- `s` is a **string object**.
- `.upper()` is a **string method** that returns the uppercase version.

7.2 Functions vs Methods

- A **function** is **independent and operates on input arguments**.
- A **method** is a function that **belongs to an object and operates on the object's internal data**.

```

1 # Function
2 def square(x):
3     return x * x
4
5 print(square(3)) # Output: 9
6
7 # Method in class
8 class Math:
9     def square(self, x):
10         return x * x
11
12 m = Math()
13 print(m.square(3)) # Output: 9

```

7.3 Defining Classes in Python

- A class is a blueprint that groups related data and methods.

```
1 class Person:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def say_hello(self):
7         print(f"Hello, my name is {self.name} and I'm {self.age} years old.")
```

Explanation:

- `__init__()` is a **constructor**, automatically called when a new object is created.
- `self` refers to the **current object instance**.

7.4 Creating Objects (Instantiation)

```
1 p1 = Person("Alice", 30)
2 p1.say_hello() # Output: Hello, my name is Alice and I'm 30 years old.
```

- **Each instance is independent** and holds its own data.

7.5 Instance Variables and Methods

- **Instance Variables**
 - Defined in `__init__` using `self`.
 - Each object has unique values.

```
1 print(p1.name) # Output: Alice
2 p1.age = 31
3 print(p1.age) # Output: 31
```

- **Instance Methods**
 - Must have `self` as the first parameter.

```
1 class Counter:
2     def __init__(self):
3         self.count = 0
4
5     def increment(self):
6         self.count += 1
7
8 c = Counter()
9 c.increment()
10 print(c.count) # Output: 1
```

7.6 Static Methods

- Do not depend on instance data.

- Defined using `@staticmethod`.
- Essentially **a normal function**, but **logically belongs to a class**.

```
1 class MathUtils:
2     @staticmethod
3     def add(a, b):
4         return a + b
5
6 print(MathUtils.add(3, 5)) # Output: 8
```

If you omit `@staticmethod` and still define the method in a class, **Python will treat it as an instance method by default** and force a `self` parameter to be passed:

```
1 class MathUtils:
2     def add(a, b): # No self is written
3         return a + b
4
5 m = MathUtils()
6 print(m.add(3, 5))
7 # TypeError: add() missing 1 required positional argument: 'b'
```

The above code will throw an error because Python tries to implicitly pass an instance as the first argument, but we haven't defined `self`.

Solution:

- The correct way to write it is to add `@staticmethod`
- Or define the function outside the class

If we omit `@staticmethod` but still want to use this function, we can **directly call it using class name**. (method using in class)

```
1 class MathUtils:
2     def add(a, b): # No self is written
3         return a + b
4
5 print(MathUtils.add(3, 5)) # Output: 8
```

7.7 Class Methods (Optional)

- A class method receives the **class itself** as the first argument (commonly named `cls`), **instead of an instance** (`self`).
- It can **access or modify class-level variables**, but cannot access instance-level attributes directly.
- Declared using the `@classmethod` decorator.

```
1 class Dog:
2     species = "Canine" # Class variable
3
4     @classmethod
5     def get_species(cls):
6         return cls.species
```

Calling the method:

```
1 | print(Dog.get_species()) # Output: Canine
```

7.8 Comparison: Instance vs Class vs Static Methods

Feature	Instance Method (<code>def method(self)</code>)	Class Method (<code>@classmethod</code>)	Static Method (<code>@staticmethod</code>)
First parameter	<code>self</code> (object)	<code>cls</code> (class)	None
Access instance vars	✓	✗	✗
Access class vars	✓	✓	✗
Requires an object	✓	✗	✗
Usage	Acts on object data	Acts on class state / factory	Utility/helper functions

7.9 Special Methods: `__str__`, `__repr__` and `__eq__`

1. `__str__` Method: User-Friendly Representation

- Purpose:
 - Defines how an object is displayed when passed to `print()` or `str()` — i.e., a **readable string format** for end-users.
- Use case:
 - Designed to return a user-friendly, clean string. It's intended for readable output.

Example:

```
1 | class Animal:
2 |     def __init__(self, name):
3 |         self.name = name
4 |
5 |     def __str__(self):
6 |         return f"This is an animal named {self.name}."
7 |
8 | a = Animal("Tiger")
9 | print(a)           # Output: This is an animal named Tiger.    --> __str__
10 | print(str(a))      # Output: This is an animal named Tiger.    --> __str__
```

If `__str__` is not defined, printing the object would return something like:

```
1 | <__main__.Animal object at 0x10c8fa1f0>
```

2. `__repr__` Method: Developer-Friendly Representation

- Purpose:
 - Defines how an object is represented when passed to `repr()` or typed directly into an interactive interpreter

— i.e., an **unambiguous and official representation**.

- Use case:
 - Meant for developers. Ideally, it should return a string that, when passed to `eval()`, would recreate the same object (if possible).

Example:

```
1 class Animal:
2     def __init__(self, name):
3         self.name = name
4
5     def __repr__(self):
6         return f"Animal(name='{self.name}')"
7
8 a = Animal("Tiger")
9 print(repr(a)) # Output: Animal(name='Tiger') --> __repr__
```

In an interactive Python shell, just typing `a` would also output:

```
1 Animal(name='Tiger')
```

3. `__eq__` Method: Defining Equality Between Objects

- Purpose:
 - The `__eq__` method defines what it means for two objects to be considered **equal** when using the `==` operator.
- Use Case:
 - You can override this method to specify how two instances should be compared by **value** (rather than by default memory identity).

By default, Python uses object **identity** (i.e., **memory address**) to evaluate equality.

```
1 class Animal:
2     def __init__(self, name):
3         self.name = name
4
5 a1 = Animal("Tiger")
6 a2 = Animal("Tiger")
7
8 print(a1 == a2) # Output: False — different objects in memory
```

Even though the `name` attribute is the same, the objects are not equal.

You can define your own `__eq__` method to compare the **internal attributes** of objects.

Example:

```
1 class Animal:
2     def __init__(self, name):
3         self.name = name
4
5     def __eq__(self, other):
6         if isinstance(other, Animal):
7             return self.name == other.name
8         return False
```

```

9
10 a1 = Animal("Tiger")
11 a2 = Animal("Tiger")
12 a3 = Animal("Lion")
13
14 print(a1 == a2) # Output: True
15 print(a1 == a3) # Output: False

```

```

1 def __eq__(self, other):

```

- `self`: the current object.
- `other`: the object to compare against.

7.10 Class Variables vs Instance Variables

- **Class Variables**
 - Shared **across all objects**.
 - Defined **outside any methods**.

```

1 class Person:
2     species = "Human" # Class variable
3
4     def __init__(self, name):
5         self.name = name # Instance variable
6
7 p1 = Person("John")
8 p2 = Person("Jane")
9 print(p1.species) # Output: Human
10 print(p2.species) # Output: Human

```

7.11 Module Import and Driver Programs

```

1 # Suppose person.py defines the Person class
2 from person import Person
3
4 p = Person("Alice", 22)
5 p.say_hello()

```

- You can import the class in another script and instantiate objects.

7.12 Getters and Setters

1. What Are Getters and Setters?

Getters and **Setters** are methods used in object-oriented programming to **access** and **modify** the private or protected attributes of a class. Although Python allows direct attribute access, using getters and setters enables **encapsulation** and **controlled data management**.

2. Why Use Getters and Setters?

Direct access to attributes can be **risky** because:

- It allows **modification without validation**.
- It limits flexibility for future changes.
- It doesn't allow logging, constraints, or other side-effects.

Getters and setters help enforce **data validation**, **security**, and **flexibility**.

3. Basic Syntax in Python

```
1 class Person:
2     def __init__(self, name, age):
3         self._name = name           # _name and _age are protected attributes
4         self._age = age
5
6     # Getter for age
7     def get_age(self):
8         return self._age
9
10    # Setter for age
11    def set_age(self, new_age):
12        if new_age >= 0:
13            self._age = new_age
14        else:
15            print("Invalid age. Age cannot be negative.")
16
17    p = Person("Alice", 25)
18
19    print(p.get_age()) # Output: 25
20
21    p.set_age(30)
22    print(p.get_age()) # Output: 30
23
24    p.set_age(-5)      # Output: Invalid age. Age cannot be negative.
```

4. Why Use an Underscore (_) Prefix?

In Python, a single leading underscore (like `_age`) indicates a **protected attribute** by convention. It doesn't enforce privacy but signals "internal use only".

5. The Pythonic Way: `@property` and `@setter`

Python provides decorators `@property` and `@<property>.setter` to define **readable and writable attributes** in a more elegant way.

```
1 class Person:
2     def __init__(self, name, age):
3         self._name = name
4         self._age = age
5
6     @property
7     def age(self): # Getter
8         return self._age
9
10    @age.setter
11    def age(self, value): # Setter
```

```
12         if value >= 0:
13             self._age = value
14         else:
15             print("Invalid age.")
16
17 p = Person("Alice", 25)
18
19 print(p.age)      # Output: 25 ← calls the getter
20 p.age = 30        # ← calls the setter
21 print(p.age)      # Output: 30
22 p.age = -1        # Output: Invalid age.
```

8 More Flow Control

8.1 Advanced Flow Control: `break`, `continue`, `return`, `raise`, `try-except`, `assert`

8.1.1 `break`: Exit a loop early

Explanation:

- `break` is a reserved keyword.
- It is used to **immediately exit the current loop** (whether `for` or `while`), regardless of the loop condition.
- Commonly used when you want to terminate a loop based on a specific condition.

Example:

```
1 for i in range(10):
2     if i == 5:
3         break
4     print(i) # Output: 0 to 4 (excluding 5)
```

Output:

```
1 0
2 1
3 2
4 3
5 4
```

Notes:

- `break` only exits the innermost loop.
- It's useful in nested structures where an early exit is required.

8.1.2 `continue`: Skip the current iteration

Explanation:

- `continue` is a reserved keyword.
- It is used to **skip the rest of the current iteration** and jump to the next loop cycle.

Example:

```
1 for i in range(5):
2     if i == 2:
3         continue
4     print(i) # Output: 0 1 3 4 (skips 2)
```

Output:

```
1 0
2 1
3 3
4 4
```

Notes:

- Ideal for skipping invalid inputs or values that don't meet a condition.

8.1.3 `return`: End function and return value

Explanation:

- `return` ends function execution and optionally sends a value back to the caller.
- If not specified, the function returns `None` by default.
- A function can have multiple `return` statements, but only the first executed one takes effect.

Example 1: Basic return

```
1 def add(a, b):
2     return a + b
3
4 print(add(2, 3)) # Output: 5
```

Example 2: Conditional return

```
1 def is_even(n):
2     if n % 2 == 0:
3         return True
4     return False
5
6 print(is_even(4)) # Output: True
7 print(is_even(5)) # Output: False
```

Example 3: Returning multiple values (tuple)

```
1 def calc(a, b):
2     return a + b, a * b
3
4 result = calc(3, 4)
5 print(result) # Output: (7, 12)
```

8.1.4 `raise`: Manually trigger an exception

Explanation:

- `raise` is used to **manually generate an exception**.
- This is useful for alerting errors when certain conditions are violated.

Example:

```
1 def divide(a, b):
2     if b == 0:
3         raise ValueError("Cannot divide by zero")
4     return a / b
5
6 # divide(4, 0) # Uncommenting this line will raise ValueError
```

8.1.5 `try-except`: Handle exceptions gracefully

Explanation:

- Code that might cause an error is placed inside the `try` block.
- Errors are caught and handled in the `except` block.
- Useful for writing robust programs that don't crash on minor issues.

Example:

```
1 try:
2     result = 10 / 0
3 except ZeroDivisionError:
4     print("Division by zero is not allowed.") # This line is printed
```

Output:

```
1 Division by zero is not allowed.
```

Extended Example:

```
1 try:
2     int("abc")
3 except ValueError as e:
4     print("Caught error:", e)
5 finally:
6     print("Execution complete.") # Always runs
```

8.1.6 `assert`

`assert` is a built-in Python statement used to **debug** your program and **check assumptions**. When the program reaches an `assert` statement:

- If the expression is `True`, the program continues normally.
- If the expression is `False`, it raises an `AssertionError` (optionally with a custom error message), indicating that something went wrong or a logic condition wasn't met.

Syntax

```
1 | assert <expression>, <optional error message>
```

- `<expression>`: A condition that must be `True`.
- `<optional error message>`: A custom message that appears with the `AssertionError`.

Example 1: A passing assertion

```
1 | x = 5
2 | assert x > 0 # Passes silently, no error
```

Example 2: Failing assertion with a message

```
1 | x = -2
2 | assert x > 0, "x must be positive"
3 | # Raises: AssertionError: x must be positive
```

Example 3: Assertion inside a function

```
1 | def square_root(x):
2 |     assert x >= 0, "Input must be non-negative"
3 |     return x ** 0.5
4 |
5 | print(square_root(9)) # Output: 3.0
6 | # print(square_root(-1)) # Raises AssertionError: Input must be non-negative
```

8.2 for Loops and the range Function

8.2.1 for loop

Explanation:

- Used to iterate over iterable objects (like lists, strings, or tuples).
- Automatically handles initialization, condition check, and update.

Example:

```
1 | for char in "abc":
2 |     print(char)
3 | # Output:
4 | # a
5 | # b
6 | # c
```

8.2.2 range(start, stop, step) function

Syntax:

- `range(stop)` → from 0 to `stop - 1`
- `range(start, stop)` → from `start` to `stop - 1`

- `range(start, stop, step)` → increment or decrement using `step`

Example:

```
1 for i in range(1, 6):
2     print(i) # Output: 1 to 5
3
4
5 for i in range(10, 1, -2):
6     print(i) # Output: 10, 8, 6, 4, 2
```

8.3 Nested Structures and Combined Control Flow

8.3.1 Nested loop with `break`

Example:

```
1 for i in range(3): # Outer loop
2     for j in range(5): # Inner loop
3         if j == 2:
4             break
5         print(f"i={i}, j={j}")
```

Output:

```
1 i=0, j=0
2 i=0, j=1
3 i=1, j=0
4 i=1, j=1
5 i=2, j=0
6 i=2, j=1
```

8.4 Built-in Exceptions You Can Raise in Python

You can use the `raise` statement in Python to manually raise **built-in exceptions** or **custom exceptions**. All exceptions in Python inherit from the base class `BaseException`, and most common ones derive from `Exception`.

Here's a list of **commonly raised built-in exceptions**, grouped by typical usage:

Exception Class	When to Use / Description
<code>ValueError</code>	Argument type is valid, but the value is inappropriate (<code>int("abc")</code>)
<code>TypeError</code>	Mismatched types (<code>1 + "a"</code>)
<code>IndexError</code>	Index out of range (e.g., <code>my_list[100]</code>)
<code>KeyError</code>	Key not found in a dictionary
<code>AttributeError</code>	Object has no such attribute
<code>NameError</code>	Variable is not defined
<code>ZeroDivisionError</code>	Division by zero
<code>FileNotFoundError</code>	File does not exist
<code>PermissionError</code>	Permission denied when accessing a file
<code>ImportError</code>	Import failed
<code>ModuleNotFoundError</code>	Module cannot be found (subclass of <code>ImportError</code>)
<code>AssertionError</code>	Assertion failed (usually triggered by <code>assert</code>)
<code>StopIteration</code>	No more elements in iterator
<code>RuntimeError</code>	General runtime error
<code>NotImplementedError</code>	Indicates a method is unimplemented (for abstract classes)
<code>RecursionError</code>	Exceeded max recursion depth
<code>MemoryError</code>	Out of memory
<code>OverflowError</code>	Numeric result too large
<code>UnicodeError</code>	Encoding/decoding issues
<code>EOFError</code>	Reached end of file/input stream
<code>KeyboardInterrupt</code>	User interrupted execution with Ctrl+C

9. File Input/Output

9.1 Introduction: Why is File I/O Important?

- Programs often need to *store* and *load* data across time.
- File I/O enables:
 - Persistent storage beyond the program’s execution;
 - Access to large or historical datasets;
 - Logging, backups, and configuration files;
 - Real-world applications like reading reports, writing logs, or storing user data.

2. Standard Input/Output and File I/O in Python

- **Standard Input and Output**

```
1 name = input("Enter your name: ") # User types something in the terminal
2 print(f"Hello, {name}") # Output example: Hello, Alice
```

- **File I/O Basics**

Use the `open()` function with a mode string to create a file object:

- `'r'`: read mode (default)
- `'w'`: write mode (overwrites file)
- `'a'`: append mode
- `'rb'` / `'wb'`: binary read/write

3. Reading Files

3.1 `read()`: Reads the entire file as a string

```
1 f = open("example.txt", "r")
2 contents = f.read()
3 print(contents) # Output the whole content
4 f.close()
```

3.2 `readline()`: Reads one line at a time

```
1 f = open("example.txt", "r")
2 line1 = f.readline()
3 line2 = f.readline()
4 print(line1.strip()) # Remove newline
5 print(line2.strip())
6 f.close()
```

3.3 `readlines()`: Reads all lines into a list

```
1 f = open("example.txt", "r")
2 lines = f.readlines()
3 print(lines) # Example output: ['Line1\n', 'Line2\n']
4 f.close()
```

4. Writing Files

4.1 `write()`: Writes a string to the file

Writes a string to a file, **returning the number of characters written**.

```

1 f = open("output.txt", "w")
2 f.write("Hello, world!\n")
3 f.write("This is another line.\n")
4
5 chars_written = f.write("Hello\n")
6 print(chars_written) # output: 6
7 f.close()

```

4.2 `writelines()`: Writes a list of strings

No line breaks are added automatically

```

1 lines = ["First line\n", "Second line\n"]
2 f = open("output.txt", "w")
3 f.writelines(lines)
4 f.close()

```

Summary: Input/Output

– Inputs:

- Standard input – `input()`
- Command line arguments
- File inputs – read from text file
 - `read()` -> str
 - `readline()` -> str
 - `readlines()` -> list

– Output:

- Standard output – `print()`
- File output – write to text file
 - `write()` -> int
 - `writelines()` -> None

4.3 Writing to a file with `print()`

```

1 with open("output.txt", "w") as f:
2     print("Hello, world!", file=f)

```

• Explanation:

- The `file=f` parameter redirects the output to a file.
- `print()` essentially outputs a string to a "stream object" (such as the screen or a file).

5. Closing Files

Using `.close()`

```

1 f = open("file.txt", "r")
2 # Process the file
3 f.close()

```

Recommended: Use `with` to auto-close

```
1 with open("example.txt", "r") as f:
2     contents = f.read()
3     print(contents)
4 # File is automatically closed after the block
```

6. File Paths

Absolute vs Relative Path

```
1 # Relative path (from current directory)
2 with open("data/info.txt", "r") as f:
3     print(f.readline())
4
5 # Absolute path (starts from root)
6 with open("/home/user/data/info.txt", "r") as f:
7     print(f.readline())
```

7. The `os` Module for File Operations

```
1 import os
2
3 print(os.getcwd())      # Print current directory
4 print(os.listdir("."))  # List contents of current directory
```

8. Exception Handling: `try-except-finally`

- Basic Structure

```
1 try:
2     f = open("missing.txt", "r")
3     content = f.read()
4 except FileNotFoundError:
5     print("The file was not found.")
6 finally:
7     print("This always runs.") # Regardless of whether an exception occurs, the code in the
                                # finally block will always be executed.
```

- Common Exceptions

Type	Example
<code>FileNotFoundError</code>	File not found
<code>ValueError</code>	e.g. <code>int("abc")</code>
<code>ZeroDivisionError</code>	e.g. <code>1 / 0</code>
<code>IndexError</code>	Invalid index (e.g. <code>sys.argv[99999]</code>)
<code>TypeError</code>	Type mismatch (e.g., <code>2 + "2"</code>)
<code>NameError</code>	Using undefined variable (e.g. <code>print(hello)</code>)
<code>SyntaxError</code>	e.g. <code>print("Hello)</code>

String Methods: split() and strip()

1. split() Method

Purpose:

The `split()` method divides a string into a list using a **specified delimiter**.

Syntax:

```
1 | str.split(separator, maxsplit)
```

- `separator`: Optional. Specifies the delimiter. If omitted, whitespace is used.
- `maxsplit`: Optional. Specifies the maximum number of splits.

Example 1: Split by whitespace (default)

```
1 | text = "Hello Python World"
2 | words = text.split()
3 | print(words) # Output: ['Hello', 'Python', 'World']
```

Example 2: Split by a specific character

```
1 | line = "name,age,gender"
2 | tokens = line.split(",")
3 | print(tokens) # Output: ['name', 'age', 'gender']
```

Example 3: Limit number of splits

```
1 | data = "a:b:c:d"
2 | print(data.split(":", 2)) # Output: ['a', 'b', 'c:d']
```

2. strip() Method

Purpose:

The `strip()` method removes **leading and trailing whitespace or specified characters** from a string.

Syntax:

```
1 | str.strip(chars)
```

- `chars`: Optional. A string specifying the **set of characters** to remove. Not a substring.
- **By default**, it removes whitespace, newline characters (`\n`), and tab characters (`\t`).

Example 1: Remove leading/trailing spaces

```
1 | text = "    Hello World!    "  
2 | print(text.strip()) # Output: "Hello World!"
```

Example 2: Remove newline characters

```
1 | line = "line content\n"  
2 | print(line.strip()) # Output: "line content"
```

Example 3: Remove specific characters

```
1 | data = "***Hello***"  
2 | print(data.strip("*")) # Output: "Hello"
```

- **Note**
 - `strip("*")` will repeatedly remove all characters that appear in `"*"` from the beginning and end of string until it encounters a character that is not in `chars`.

Note: `strip()` matches individual characters, not substrings

Example 1:

```
1 | data = "xyxHelloxy"  
2 | print(data.strip("xy")) # Output: "Hello"
```

Explanation:

- Both `x` and `y` are in the string `"xy"`.
- From the left: `x`, `y`, and `x` are removed.
- From the right: `o`, `x`, `y` are examined — `x` and `y` are removed.
- The middle `"Hello"` is untouched.

Example 2:

```
1 | data = "abcHelloabc"  
2 | print(data.strip("abc")) # Output: "Hello"
```

Although `"abc"` is a substring, `strip()` does **not** treat it as a single unit. It removes:

- All characters at the start and end that are either `a`, `b`, or `c`.

10. Testing

10.1 Testing Fundamentals

1. Definition of a Test

A **test** is a procedure to evaluate the **quality, performance, or reliability** of something—especially before widespread use.

2. Debugging with Print Statements

```
1 def add(a, b):
2     print(f"Adding {a} and {b}") # Debugging output
3     return a + b
4
5 result = add(3, 4) # Output: Adding 3 and 4
6 print(result)     # Output: 7
```

Explanation:

- `print()` is the simplest debugging tool.
- Use it to observe intermediate variable states.
- For more advanced projects, consider using `logging` instead.

3. Test Driven Development (TDD)

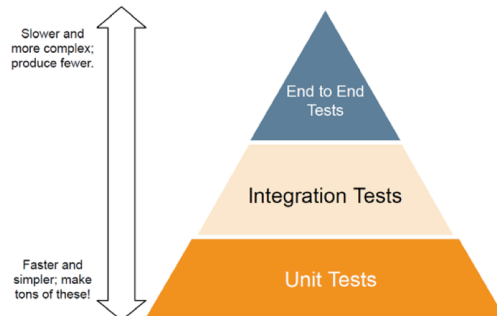
- **This is a way of software development that requires you to write the tests for a given function first, before you even write the function!**
- It makes you think exactly what you want your function(s) to do in terms of:
 - What the normal functionality should be?
 - What should happen given “bad” input (e.g. passing a negative number to a function that expects positive numbers).
- Should test whether you are on the right track throughout development and not only at the end.

4. Types of Tests: The Testing Pyramid

- **End-to-end Tests**
 - Full functionality of system.
- **Integration Tests**
 - Examines how different components of a system interact with one another.
- **Unit Tests**
 - Fast and simple tests that target the smallest possible components of a system.

Types of Tests: The Testing Pyramid

- End-to-end Tests
 - Full functionality of system.
- Integration Tests
 - Examines how different components of a system interact with one another.
- Unit Tests
 - Fast and simple tests that target the *smallest* possible components of a system.



10.2 Designing Test Cases

1. Types of Test Cases

- **Positive Case:** Input is valid; system should behave normally.
- **Negative Case:** Input is invalid; system should handle gracefully.
- **Edge Case:** Input is at boundary of valid range (e.g., 0, empty list).
- **Corner Case:** Multiple extreme values at once (e.g., max load + wind).

2. Example: `is_even()` Function

```
1 def is_even(n):
2     return isinstance(n, int) and n % 2 == 0
3
4 print(is_even(2))      # Output: True (positive case)
5 print(is_even(5))      # Output: False (positive case)
6 print(is_even(5.5))    # Output: False (negative case)
7 print(is_even("Hi"))   # Output: False (negative case)
8 print(is_even([]))     # Output: False (negative case)
9 print(is_even(-4))     # Output: True (edge case)
```

Explanation:

- First, `isinstance(n, int)` checks if input is an integer.
- `% 2 == 0` checks evenness.

10.3 `assert` Statement & Manual Testing

1. Using `assert` for Debugging

```

1 def divide(a, b):
2     assert b != 0, "Divider cannot be zero" # Raise error if b is 0
3     return a / b
4
5 print(divide(10, 2)) # Output: 5.0
6 # print(divide(10, 0)) # AssertionError: Divider cannot be zero

```

Explanation:

- `assert` checks a condition.
- If `False`, it raises an `AssertionError`.
- Generally used during development—not recommended for production.

2. Problem with Manual Regression Testing

- Every time you change code, you need to rerun all previous tests.
- Tedious and error-prone — solution: **automated testing**.

10.4 Writing a Test Driver

Test Driver Example

```

1 # math_ops.py
2 def square(n):
3     return n * n

```

```

1 # test_driver.py
2 import math_ops
3
4 def test_square_positive():
5     assert math_ops.square(2) == 4
6
7 def test_square_negative():
8     assert math_ops.square(-3) == 9
9
10 def test_square_zero():
11     assert math_ops.square(0) == 0
12
13 test_square_positive()
14 test_square_negative()
15 test_square_zero()
16 print("All tests passed.") # Only prints if all assertions succeed

```

Explanation:

- Keep test code in a separate file.
- Each test case is wrapped in its own function.

10.5 The `unittest` Framework

1. Basic Structure of `unittest`

```

1  import unittest
2
3  def is_even(n):
4      return isinstance(n, int) and n % 2 == 0
5
6  class TestIsEven(unittest.TestCase):
7
8      def test_even_number(self):
9          self.assertTrue(is_even(2))
10
11     def test_odd_number(self):
12         self.assertFalse(is_even(3))
13
14     def test_non_integer(self):
15         self.assertFalse(is_even("Hello"))
16
17 if __name__ == '__main__':
18     unittest.main()

```

Command to run:

```
1 | python3 -m unittest test_script.py
```

Explanation:

- Class must inherit from `unittest.TestCase`.
- Each test method starts with `test_`.
- Use assertion methods to check expected outcomes.

2. Common Assertion Methods in `unittest`

Method	Description
<code>assertEqual(a, b)</code>	Checks that <code>a == b</code>
<code>assertNotEqual(a, b)</code>	Checks that <code>a != b</code>
<code>assertTrue(x)</code>	Checks that <code>x is True</code>
<code>assertFalse(x)</code>	Checks that <code>x is False</code>
<code>assertIsInstance(x, T)</code>	Checks <code>x</code> is instance of type <code>T</code>
<code>assertGreater(a, b)</code>	Checks <code>a > b</code>
<code>assertGreaterEqual(a, b)</code>	Checks <code>a >= b</code>
<code>assertAlmostEqual(a, b)</code>	Checks <code>round(a-b, 7) == 0</code>
<code>assertNotAlmostEqual(a, b)</code>	Checks <code>round(a-b, 7) != 0</code>

Example:

```
1 self.assertEqual(3 * 2, 6)
2 self.assertNotEqual(3 + 4, 10)
3 self.assertTrue(is_even(4))
4 self.assertFalse(is_even(5))
5 self.assertIsInstance(5, int)
```

10.6 Using `doctest` Module

1. Testing with Docstrings and `doctest`

`doctest` is a module in the Python standard library that is **used to write test code in documentation strings (docstrings)** and automatically run these tests to check whether the output is consistent with expectations.

```
1 def multiply(a, b):
2     """
3     Multiply two numbers and return the result.
4
5     >>> multiply(2, 3)
6     6
7     >>> multiply(-1, 5)
8     -5
9     """
10    return a * b
11
12 if __name__ == "__main__":
13     import doctest
14     doctest.testmod()
```

Explanation:

- Docstrings include examples that mimic interactive Python sessions.
- `doctest.testmod()` runs these examples and validates the output.
- **No output = all tests passed. Output means something failed.**

10.7 Testing Terminology

• Key Concepts

- **Test case:** A single, specific test (e.g., test `is_even(2)`).
- **Test suite:** A group of test cases run together.
- **Test runner:** A tool that runs tests and reports results (e.g., `unittest.main()`).

10.8 White Box and Black Box Testing

Black Box Testing

Black Box Testing treats the software as a “black box”—you do **not know the internal implementation**. You only test whether the system behaves correctly by giving inputs and checking the outputs.

- Focus: **What** the program does (external behavior).
- Common in **functional testing, system testing, and user acceptance testing**.

White Box Testing

White Box Testing (also known as structural testing) assumes **full knowledge of the internal workings** of the program. You design tests based on code logic, branches, loops, and statements.

- Focus: **How** the program does what it does (internal structure).
- Common in **unit testing, logic path testing, regression testing**.

White Box vs. Black Box

Black Box Testing	White Box Testing
Focuses on what the program should do.	Focuses on how the program does it.
No access to the code (blind test).	Writing tests based on seeing the code.
Good for catching specification mismatches.	Good for targeting specific control flow, logic, or lines suspected to be incorrect.

11 Recursion

11.1 What is Recursion?

Definition:

- **Recursion** refers to a function that **calls itself** within its definition.
- It is a **problem-solving technique** where a large problem is broken down into smaller instances of the same problem.

Example:

```
1 def greet():
2     print("Hello")
3     greet() # calls itself
4
5 greet()
```

This code will cause **infinite recursion**, continuously printing "Hello" until it crashes. To avoid this, we must define a **base case** to stop the recursion.

11.2 Two Core Parts of Recursion

1. Base Case

- The simplest instance of the problem that does **not require recursion**.
- Prevents infinite recursion.

2. Recursive Case

- The part where the function **calls itself** with a smaller or simpler argument, gradually approaching the base case.

Example: Sum of First n Natural Numbers

```
1 def sum_n(n):
2     if n == 1:          # base case
3         return 1
4     return sum_n(n - 1) + n # recursive case
5
6 print(sum_n(5)) # Output: 15 (1+2+3+4+5)
```

11.3 Problems Solved Using Recursion

1. Countdown Printing

```
1 def countdown(n):
2     if n == 0: # base case
3         print("Lift off!") # Output: Lift off!
4     else:
5         print(n)
6         countdown(n - 1)
7
8 countdown(5)
9 # Output:
10 # 5
11 # 4
12 # 3
13 # 2
14 # 1
15 # Lift off!
```

2. Reverse vs Forward Printing

```
1 # Print 1 to n
2 def print_up(n):
3     if n > 0:
4         print_up(n - 1)
5         print(n)
6
7 print_up(3)
8 # Output:
9 # 1
10 # 2
11 # 3
12
13 # Print n to 1
14 def print_down(n):
15     if n > 0:
16         print(n)
17         print_down(n - 1)
18
19 print_down(3)
20 # Output:
21 # 3
22 # 2
23 # 1
```

11.4 Different Ways to Write Base Cases

All the following compute the sum of the first `n` natural numbers:

```
1 # Method 1: Base case with n == 0
2 def sum_n(n):
3     if n == 0:
4         return 0
5     return n + sum_n(n - 1)
6
7 # Method 2: Base case with n == 1
8 def sum_n(n):
9     if n == 1:
10        return 1
11    return n + sum_n(n - 1)
12
13 # Method 3: Defensive base case (n <= 0)
14 def sum_n(n):
15     if n <= 0:
16         return 0
17    return n + sum_n(n - 1)
```

11.5 Infinite Recursion

If a function **does not reach the base case**, it keeps calling itself forever, eventually causing a `RecursionError`.

```
1 def infinite():
2     return infinite() + 1
3
4 # infinite() # Uncommenting will cause RecursionError
```

11.6 Understanding Memory Use in Recursion: The Call Stack

Each recursive call creates a new **stack frame** in memory:

- This includes local variables, return addresses, etc.
- Stack frames return in reverse order (LIFO).

Example:

```
1 def trace(n):
2     print(f"Enter: {n}")
3     if n == 0:
4         print("Base case")
5     else:
6         trace(n - 1)
7     print(f"Exit: {n}")
8
9 trace(2)
10 # Output:
11 # Enter: 2
12 # Enter: 1
13 # Enter: 0
14 # Base case
```

```
15 # Exit: 0
16 # Exit: 1
17 # Exit: 2
```

11.7 Tower of Hanoi Problem

Rules:

- Only one disk can be moved at a time.
- A larger disk may not be placed on top of a smaller one.
- Use an auxiliary rod to help transfer disks.

Example (3 disks):

```
1 def hanoi(n, source, auxiliary, target):
2     if n == 1:
3         print(f"Move disk 1 from {source} to {target}")
4     else:
5         hanoi(n - 1, source, target, auxiliary)
6         print(f"Move disk {n} from {source} to {target}")
7         hanoi(n - 1, auxiliary, source, target)
8
9 hanoi(3, 'A', 'B', 'C')
10 # Output:
11 # Move disk 1 from A to C
12 # Move disk 2 from A to B
13 # Move disk 1 from C to B
14 # Move disk 3 from A to C
15 # Move disk 1 from B to A
16 # Move disk 2 from B to C
17 # Move disk 1 from A to C
```

12. Multi-dimensional Lists and Numpy Arrays

12.1 Review: Python Lists

- Lists can contain elements of different types

```
1 mixed_list = [1, "hello", 3.14, True]
2 print(mixed_list) # Output: [1, 'hello', 3.14, True]
```

- Accessing list elements using zero-based indexing

```
1 names = ["Alice", "Bob", "Charlie"]
2 print(names[0]) # Output: Alice
3 print(names[2]) # Output: Charlie
```

- Getting the length of a list

```
1 print(len(names)) # Output: 3
```

12.2 Multi-dimensional Lists

- **Dimensional Concepts**

- Scalar: a single value, e.g., `x = 5`
- 1D Vector: `[1, 2, 3]`
- 2D Matrix: `[[1, 2], [3, 4]]`
- N-Dimensional: e.g., a 3D structure like `[depth][row][column]`

12.3 2D Lists

- **Creating a 2D list**

```
1 matrix = [  
2     [1, 2, 3],  
3     [4, 5, 6],  
4     [7, 8, 9]  
5 ]
```

- **Accessing elements**

```
1 print(matrix[0][0]) # Output: 1  
2 print(matrix[1][2]) # Output: 6
```

- **Traversing a 2D list**

```
1 for row in matrix:  
2     for element in row:  
3         print(element, end=' ')  
4 # Output: 1 2 3 4 5 6 7 8 9
```

- **Initialising a 2D list using list comprehension**

```
1 rows, cols = 3, 4  
2 matrix = [[0 for _ in range(cols)] for _ in range(rows)]  
3 print(matrix)  
4 # Output: [[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
```

12.4 3D Lists

- **Creating a 3D list**

```
1 tensor = [  
2     [[1, 2], [3, 4]],  
3     [[5, 6], [7, 8]]  
4 ]  
5 print(tensor[0][1][1]) # Output: 4  
6 print(tensor[1][0][0]) # Output: 5
```

- **Traversing a 3D list**

```
1 for layer in tensor:
2     for row in layer:
3         for value in row:
4             print(value, end=' ')
5 # Output: 1 2 3 4 5 6 7 8
```

12.5 NumPy Basics

- **Why use NumPy?**
 - Python lists are inefficient for numerical computation
 - NumPy supports fast operations, vectorisation, and powerful matrix operations
- **Importing NumPy**

```
1 import numpy as np
```

- **Creating arrays**

From list:

```
1 a = np.array([1, 2, 3])
2 print(a) # Output: [1 2 3]
```

Creating matrices:

```
1 matrix = np.array([[1, 2], [3, 4]])
2 print(matrix)
3 # Output:
4 # [[1 2]
5 #  [3 4]]
```

Using built-in functions:

```
1 print(np.zeros((2, 3))) # 2x3 zero matrix
2 print(np.ones((2, 2))) # 2x2 matrix filled with 1
3 print(np.empty((1, 3))) # Uninitialised 1x3 matrix
4 print(np.arange(0, 10, 2)) # Output: [0 2 4 6 8]
5 print(np.linspace(0, 1, 5)) # Output: [0.  0.25 0.5  0.75 1. ]
```

-
- **Array Attributes**

```
1 a = np.array([[1, 2, 3], [4, 5, 6]])
2
3 print(a.shape) # Output: (2, 3) → 2 rows, 3 columns
4 print(a.dtype) # Output: int64 (data type)
5 print(a.ndim) # Output: 2 (2D array)
6 print(a.size) # Output: 6 (total number of elements)
```

12.6 NumPy Array Operations

- Indexing and slicing

```
1 print(a[0, 1])    # Output: 2
2 print(a[:, 1])    # Output: [2 5] → all rows, column 1
```

- Vectorised operations

```
1 a = np.array([1, 2, 3])
2 b = np.array([4, 5, 6])
3 print(a + b)      # Output: [5 7 9]
4 print(a * b)      # Output: [ 4 10 18]
```

- Transpose and reshape

```
1 matrix = np.array([[1, 2], [3, 4]])
2 print(matrix.T)
3 # Output:
4 # [[1 3]
5 #  [2 4]]
6
7 reshaped = matrix.reshape(4, 1)
8 print(reshaped)
9 # Output:
10 # [[1]
11 #  [2]
12 #  [3]
13 #  [4]]
```

12.7 Traversing with NumPy

- Looping through 2D array

```
1 a = np.array([[1, 2], [3, 4]])
2 for i in range(a.shape[0]):
3     for j in range(a.shape[1]):
4         print(f"a[{i}][{j}] = {a[i][j]}")
5 # Output:
6 # a[0][0] = 1
7 # a[0][1] = 2
8 # a[1][0] = 3
9 # a[1][1] = 4
```

12.8 Practice Activity: Matrix Game Simulation

A simplified example of a 5x5 grid game with a player, agent, and exit:

```
1 import numpy as np
2
3 grid = np.full((5, 5), 'O') # Fill with 'O'
4 grid[2, 2] = 'N'           # Player starts in the center
5 grid[1, 4] = 'A'           # Agent at a random position
6 grid[4, 0] = 'E'           # Exit at a random position
7
8 print(grid)
```